

# 知伴--个性化学习辅助系统

V1.0

源代码

```
# -*- coding: utf-8 -*-
import asyncio
import json
import logging
import weakref
from datetime import datetime
from zoneinfo import ZoneInfo

import httpx
from backend.src.ai_core.llm_config import llm
from backend.src.ai_core.tools.knowledge import (
    search_knowledge_base, ingest_document,
    search_web_and_stage_knowledge,
    list_knowledge, update_knowledge, delete_knowledge,
)
from backend.src.ai_core.tools.portrait import read_portrait, update_portrait
from backend.src.ai_core.tools.skill import (
    read_skill, upsert_skill, list_skills, delete_skill, create_action_skill,
)
from backend.src.ai_core.tools.resource import generate_learning_resource
from backend.src.ai_core.tools.search import web_search
from backend.src.ai_core.tools.mcp_external import load_external_mcp_tools
from backend.src.ai_core.tools.image import generate_image
from backend.src.ai_core.tools.exam import generate_exam_questions
from backend.src.ai_core.tools.path import (
    list_learning_paths, get_learning_path_detail, enroll_learning_path,
    regenerate_learning_path, update_path_node, add_path_node, delete_path_node,
)
from backend.src.ai_core.tools.animation import generate_slide_animation
from backend.src.ai_core.tools.video_search import search_online_video
from backend.src.ai_core.tools.history import get_used_history
from backend.src.ai_core.tools.memory import search_memory
from backend.src.utils.prompt_loader import load_prompt
from pydantic import create_model, Field as PydanticField

try:
    from langchain_classic.agents import AgentExecutor, create_tool_calling_agent
except ModuleNotFoundError:
    from langchain.agents import AgentExecutor, create_tool_calling_agent
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.tools import StructuredTool
from langchain_core.messages import HumanMessage, AIMessage

def _inject_user_id(tool, user_id: str):
    """拷贝一个 tool，移除 user_id 参数并自动注入当前用户 ID"""
    original_coro = tool.coroutine
    if tool.args_schema:
        fields = {}
        for name, field_info in tool.args_schema.model_fields.items():
            if name != "user_id":
```

```

        fields[name] = (field_info.annotation, field_info)
        new_schema = create_model(f"{tool.name}_input", **fields) if fields else None
    else:
        new_schema = None

    desc = (tool.description or "").replace("user_id用户数字ID", "")
    desc = desc.replace(", , ", ", ").replace(", 。", ", ").replace("参数: ", ", "参数: ").strip()

    async def _scoped(**kwargs):
        kwargs["user_id"] = user_id
        return await original_coro(**kwargs)

    _scoped.__name__ = tool.name
    return StructuredTool.from_function(
        coroutine=_scoped,
        name=tool.name,
        description=desc,
        args_schema=new_schema,
    )

def _inject_chat_group_id(tool, chat_group_id: int):
    """为 get_used_history 注入当前聊天组 ID"""
    original_coro = tool.coroutine
    if tool.args_schema:
        fields = {}
        for name, field_info in tool.args_schema.model_fields.items():
            if name not in ("chat_group_id",):
                fields[name] = (field_info.annotation, field_info)
        new_schema = create_model(f"{tool.name}_scoped_input", **fields) if fields else None
    else:
        new_schema = None

    async def _scoped(**kwargs):
        kwargs["chat_group_id"] = chat_group_id
        return await original_coro(**kwargs)

    _scoped.__name__ = tool.name
    return StructuredTool.from_function(
        coroutine=_scoped,
        name=tool.name,
        description=(tool.description or ""),
        args_schema=new_schema,
    )

_MAX_HISTORY_TURNS = 20

# —— 消息分类：按需加载工具行为指南 ——

```

```

_CREATE_TRIGGERS = [
    "生成学习", "生成资料", "生成文档", "做个PPT", "做PPT", "生成PPT",
    "整理成文档", "整理成PPT", "做成文档", "做成PPT",
    "生成思维导图", "生成脑图", "做思维导图", "做脑图",
    "帮我整理", "帮我总结", "帮我生成",
    "出题", "出几道", "出一些题", "练习题", "测验", "考试模拟",
    "做几道题", "来几道题", "做题", "习题", "试卷",
    "画一张", "画个", "生成图片", "生成一张图", "配图", "插图",
    "帮我画", "帮我生成图",
    "生成动画", "播放PPT", "演示PPT", "旁白", "念给我听",
    "搜视频", "找视频", "视频教程",
]

_MANAGE_TRIGGERS = [
    "学习路径", "课程路径", "学习计划", "选课", "有哪些路径",
    "加入路径", "路径管理", "修改节点", "添加节点", "删除节点",
    "重新规划路径", "路径不合适", "路径查看",
    "skill", "Skill", "自定义提示词", "修改提示词", "设置提示词",
    "恢复默认", "升级生成", "删除skill", "创建skill",
    "动作skill", "action skill", "添加能力", "添加工具",
]

def _classify_message(message: str) -> set[str]:
    """根据用户消息判断需要加载哪些工具行为指南"""
    cats = set()
    for t in _CREATE_TRIGGERS:
        if t in message:
            cats.add("create")
            break
    for t in _MANAGE_TRIGGERS:
        if t in message:
            cats.add("manage")
            break
    return cats

# —— 工具注册表: 工具名 → 工厂函数(uid, gid) → 已注入的 LangChain Tool ——
TOOL_REGISTRY: dict[str, callable] = {
    "search_knowledge_base": lambda uid, gid: _inject_user_id(search_knowledge_base, uid),
    "ingest_document": lambda uid, gid: _inject_user_id(ingest_document, uid),
    "search_web_and_stage_knowledge": lambda uid, gid: _inject_user_id(search_web_and_stage_knowledge, uid),
    "list_knowledge": lambda uid, gid: _inject_user_id(list_knowledge, uid),
    "update_knowledge": lambda uid, gid: _inject_user_id(update_knowledge, uid),
    "delete_knowledge": lambda uid, gid: _inject_user_id(delete_knowledge, uid),
    "read_portrait": lambda uid, gid: _inject_user_id(read_portrait, uid),
    "update_portrait": lambda uid, gid: _inject_user_id(update_portrait, uid),
    "get_used_history": lambda uid, gid: _inject_chat_group_id(_inject_user_id(get_used_history, uid), gid),
}

```

```

        "search_memory": lambda uid, gid: _inject_user_id(search_memory, uid),
        "web_search": lambda uid, gid: web_search,
        "read_skill": lambda uid, gid: _inject_user_id(read_skill, uid),
        "upsert_skill": lambda uid, gid: _inject_user_id(upsert_skill, uid),
        "list_skills": lambda uid, gid: _inject_user_id(list_skills, uid),
        "delete_skill": lambda uid, gid: _inject_user_id(delete_skill, uid),
        "create_action_skill": lambda uid, gid: _inject_user_id(create_action_skill, uid),
        "generate_learning_resource": lambda uid, gid: _inject_chat_group_id(_inject_user_id(generate_learning_resource, uid), gid),
        "generate_image": lambda uid, gid: _inject_chat_group_id(_inject_user_id(generate_image, uid), gid),
        "generate_exam_questions": lambda uid, gid: _inject_chat_group_id(_inject_user_id(generate_exam_questions, uid), gid),
        "generate_slide_animation": lambda uid, gid: _inject_chat_group_id(_inject_user_id(generate_slide_animation, uid), gid),
        "search_online_video": lambda uid, gid: _inject_chat_group_id(_inject_user_id(search_online_video, uid), gid),
        "list_learning_paths": lambda uid, gid: _inject_user_id(list_learning_paths, uid),
        "get_learning_path_detail": lambda uid, gid: _inject_user_id(get_learning_path_detail, uid),
        "enroll_learning_path": lambda uid, gid: _inject_user_id(enroll_learning_path, uid),
        "regenerate_learning_path": lambda uid, gid: _inject_user_id(regenerate_learning_path, uid),
        "update_path_node": lambda uid, gid: _inject_user_id(update_path_node, uid),
        "add_path_node": lambda uid, gid: _inject_user_id(add_path_node, uid),
        "delete_path_node": lambda uid, gid: _inject_user_id(delete_path_node, uid),
    }
}

```

```

class Brain:
    _instances: weakref.WeakSet = weakref.WeakSet()

    def __init__(self, user_id: int, chat_group_id: int | None = None,
                  session_id: str | None = None, agent_id: int | None = None):
        self.user_id = user_id
        self.chat_group_id = chat_group_id
        self.session_id = session_id or f"brain_{user_id}"
        self.agent_id = agent_id
        self._agent_persona: str | None = None
        self._agent_tool_names: set[str] | None = None
        self._agent_memory_text: str = ""
        self._raw_executor = None
        self._action_tools_loaded = False
        self._agent_config_loaded = False
        self._history: list = []
        self._history_hydrated = False # 是否已从 DB 水合最近 N 轮（幂等）
        Brain._instances.add(self)

    # —— 记忆水合 ——

    async def hydrate_history(self, before_id: int | None = None):
        """冷启动时从 DB 水合最近 N 轮历史（幂等）。
```

```

配合多级记忆系统：跨会话恢复短期记忆。before_id 用于截止（如当前
消息 id），避免把正在处理的这一轮当成历史重放。
"""
if self._history_hydrated:
    return
self._history_hydrated = True
try:
    from backend.src.models.chat_history_model import ChatHistory
    qs = ChatHistory.filter(
        user_id=self.user_id, chat_group_id=self.chat_group_id or 0
    )
    if before_id:
        qs = qs.filter(id__lt=before_id)
    records = await qs.order_by("-id").limit(_MAX_HISTORY_TURNS).all()
    for r in reversed(records):
        if r.req:
            self._history.append(HumanMessage(content=r.req))
        if r.res:
            self._history.append(AIMessage(content=r.res))
except Exception:
    logging.getLogger(__name__).exception("水合历史失败 user_id=%s", self.user_id)

# —— 动态工具工厂 ——

@staticmethod
def _make_http_tool(skill: dict):
    """将 HTTP 类型的 action skill 包装成 LangChain StructuredTool"""
    config = json.loads(skill["action_config"]) if isinstance(skill["action_config"], str) else sk
    ill["action_config"]
    safe_name = skill["name"].replace("-", "_").replace(" ", "_")

    async def _handler(**kwargs):
        url = config["url"]
        for k, v in kwargs.items():
            url = url.replace(f"{{{k}}}", str(v))
        timeout = httpx.Timeout(30.0)
        async with httpx.AsyncClient(timeout=timeout) as client:
            method = config.get("method", "GET").upper()
            resp = await client.request(method, url)
            text = resp.text[:3000]
            if resp.status_code >= 400:
                return f"请求失败 (HTTP {resp.status_code}): {text}"
            return text

    _handler.__name__ = safe_name

    params_schema = config.get("params", {})
    args_schema = None
    if params_schema:

```

```

        fields = {}
        for pname, pdesc in params_schema.items():
            if isinstance(pdsc, dict):
                desc = str(pdsc.get("description") or pdsc.get("desc") or "")
            else:
                desc = str(pdsc or "")
            fields[pname] = (str, PydanticField(description=desc))
        args_schema = create_model(f"{safe_name}_input", **fields)

    return StructuredTool.from_function(
        coroutine=_handler,
        name=safe_name,
        description=skill.get("tool_description", "") or f"自定义技能: {skill['name']}",
        args_schema=args_schema,
    )

async def _load_action_tools_async(self):
    """在正确的 async 上下文中从 DB 加载 action skill"""
    from backend.src.service.skill import service as skill_service
    skills = await skill_service.list_actions(user_id=self.user_id)
    tools = []
    for s in skills:
        if s.get("action_type") != "http":
            continue
        try:
            tools.append(self._make_http_tool(s))
        except Exception:
            logging.getLogger(__name__).exception("action skill 构造失败, 已跳过: %s", s.get("name"))
    )

    return tools

# —— 热刷新 ——

@classmethod
def rebuild_for_user(cls, user_id: int):
    """创建/删除 action skill 后标记需要刷新, 下次对话时自动重建"""
    for inst in cls._instances:
        if inst.user_id == user_id:
            inst._action_tools_loaded = False
            inst._agent_config_loaded = False

    async def _load_agent_config(self):
        """Load user-defined agent config: persona, tool whitelist, memory.
        Only runs once when agent_id is set and not yet loaded."""
        if self.agent_id is None or self._agent_config_loaded:
            return
        try:
            from backend.src.service.agent.service import get as get_agent, get_memory_text
            agent_config = await get_agent(self.user_id, self.agent_id)
            if agent_config:

```

```

        self._agent_persona = agent_config.get("persona", "") or None
        tool_names = agent_config.get("tools", [])
        self._agent_tool_names = set(tool_names) if tool_names else None
        self._agent_memory_text = await get_memory_text(self.user_id, self.agent_id)
    except Exception:
        logging.getLogger(__name__).exception("加载智能体配置失败 agent_id=%s", self.agent_id)
        self._agent_persona = None
        self._agent_tool_names = None
        self._agent_memory_text = ""
    self._agent_config_loaded = True

def _build_agent(self, action_tools: list, mcp_tools: list):
    now = datetime.now(ZoneInfo("Asia/Shanghai"))
    tz_name = "Asia/Shanghai"
    date_str = now.strftime("%Y-%m-%d")
    time_str = now.strftime("%Y-%m-%d %H:%M:%S")
    current_time_context = (
        f"\n\n### Current Time Anchor\n"
        f"- Date: {date_str}\n"
        f"- Time: {time_str}\n"
        f"- Timezone: {tz_name}\n"
        f"- Always use this date as reference for time-sensitive queries.\n"
    )

    if self._agent_persona:
        system_prompt = (
            self._agent_persona
            + current_time_context
            + ((("\n" + self._agent_memory_text) if self._agent_memory_text else "")
            + "\n\n{path_context}\n\n{portrait_context}\n\n{memory_context}"
            + "\n\n## Output Rules\n"
            + "- Use Markdown for formatting, NOT raw HTML tags.\n"
            + "- Wrap inline math in  $...$  and display math in 
$$...$$
.\n"
            + "- Never output <br>, <div>, <span> or other HTML tags.\n"
        )
    else:
        system_prompt = load_prompt("chat/unified") + current_time_context

    prompt = ChatPromptTemplate.from_messages([
        ("system", system_prompt),
        MessagesPlaceholder(variable_name="history"),
        ("human", "{input}"),
        MessagesPlaceholder(variable_name="agent_scratchpad"),
    ])

    uid = str(self.user_id)
    gid = self.chat_group_id or 0

    if self._agent_tool_names is not None:
        tool_names_to_load = self._agent_tool_names

```



```

        else:
            tool_names_to_load = set(TOOL_REGISTRY.keys())

        tools = []
        for name in tool_names_to_load:
            factory = TOOL_REGISTRY.get(name)
            if factory:
                tools.append(factory(uid, gid))

        tools.extend(_inject_user_id(t, uid) for t in action_tools)
        tools.extend(mcp_tools)

        agent = create_tool_calling_agent(llm=llm, prompt=prompt, tools=tools)
        max_iters = max(8, len(tools) * 2)
        self._raw_executor = AgentExecutor(
            agent=agent, tools=tools,
            verbose=True, handle_parsing_errors=True, max_iterations=max_iters,
        )

    def _load_tool_guides(self, message: str) -> str:
        """根据消息内容按需加载工具行为指南；未命中则返回空字符串"""
        cats = _classify_message(message)
        parts: list[str] = []
        if "create" in cats:
            parts.append(load_prompt("chat/guide_create"))
        if "manage" in cats:
            parts.append(load_prompt("chat/guide_manage"))
        return "\n".join(parts)

    async def _ensure_action_tools(self):
        """首次调用或 rebuild_for_user 后，异步加载 agent 配置、action tools 并重建 agent"""
        if self._action_tools_loaded:
            return
        await self._load_agent_config()
        try:
            action_tools = await self._load_action_tools_async()
        except Exception:
            logging.getLogger(__name__).exception("加载 action tools 失败")
            action_tools = []
        try:
            mcp_tools = await load_external_mcp_tools()
        except Exception:
            logging.getLogger(__name__).exception('Failed to load MCP tools')
            mcp_tools = []
        self._build_agent(action_tools, mcp_tools)
        self._action_tools_loaded = True

    async def chat(self, message: str, resource_context: str = "", path_context: str = "", portrait_co
ntext: str = "", memory_context: str = "") -> str:
        await self._ensure_action_tools()

```

```

        tool_guides = self._load_tool_guides(message)
        response = await self._raw_executor.ainvoke({
            "input": message,
            "history": list(self._history),
            "current_user_id": str(self.user_id),
            "resource_context": resource_context,
            "path_context": path_context,
            "portrait_context": portrait_context,
            "memory_context": memory_context,
            "tool_guides": tool_guides,
        })
        self._history.append(HumanMessage(content=message))
        self._history.append(AIMessage(content=response["output"]))
        if len(self._history) > _MAX_HISTORY_TURNS * 2:
            self._history = self._history[-_MAX_HISTORY_TURNS * 2:]
        return response["output"]

    async def stream(self, message: str, resource_context: str = "", path_context: str = "", portrait_
context: str = "", memory_context: str = ""):
        """逐 token 流式输出 — 包含工具调用事件, 工具执行期间自动心跳保活"""
        await self._ensure_action_tools()

        full_response = ""
        tool_running = False
        tool_guides = self._load_tool_guides(message)

    async def _stream_events(version: str):
        nonlocal tool_running
        agen = self._raw_executor.astream_events(
            {
                "input": message,
                "history": list(self._history),
                "current_user_id": str(self.user_id),
                "resource_context": resource_context,
                "path_context": path_context,
                "portrait_context": portrait_context,
                "memory_context": memory_context,
                "tool_guides": tool_guides,
            },
            version=version,
        )
        while True:
            try:
                event = await asyncio.wait_for(agen.__anext__(), timeout=30 if tool_running else 1
20)

            except asyncio.TimeoutError:
                yield {"type": "keepalive"}
                continue
            except StopAsyncIteration:
                break

```

```

        yield event

    try:
        async for event in _stream_events("v2"):
            kind = event.get("event", "")

            if kind == "on_tool_start":
                tool_running = True
                tool_name = event.get("name", "")
                yield {"role": "tool", "type": "tool_start", "tool": tool_name}

            elif kind == "on_tool_end":
                tool_running = False
                tool_name = event.get("name", "")
                tool_output = event.get("data", {}).get("output", "")
                if isinstance(tool_output, str) and len(tool_output) > 500:
                    tool_output = tool_output[:500] + "..."
                yield {"role": "tool", "type": "tool_end", "tool": tool_name, "output": str(tool_o

utput)}}

            elif kind == "on_chat_model_stream":
                chunk = event.get("data", {}).get("chunk")
                if chunk:
                    content = getattr(chunk, "content", None)
                    if content:
                        full_response += content
                        yield {"role": "assistant", "type": "chunk", "content": content}
    except (TypeError, NotImplementedError):
        async for event in _stream_events("v1"):
            kind = event.get("event", "")

            if kind == "on_tool_start":
                tool_running = True
                tool_name = event.get("name", "")
                yield {"role": "tool", "type": "tool_start", "tool": tool_name}

            elif kind == "on_tool_end":
                tool_running = False
                tool_name = event.get("name", "")
                tool_output = event.get("data", {}).get("output", "")
                if isinstance(tool_output, str) and len(tool_output) > 500:
                    tool_output = tool_output[:500] + "..."
                yield {"role": "tool", "type": "tool_end", "tool": tool_name, "output": str(tool_o

utput)}}

            elif kind == "on_chat_model_stream":
                chunk = event.get("data", {}).get("chunk")
                if chunk:
                    content = getattr(chunk, "content", None)
                    if content:

```

```

        full_response += content
        yield {"role": "assistant", "type": "chunk", "content": content}

    self._history.append(HumanMessage(content=message))
    self._history.append(AIMessage(content=full_response))
    if len(self._history) > _MAX_HISTORY_TURNS * 2:
        self._history = self._history[-_MAX_HISTORY_TURNS * 2:]
"""
LangGraph 双智能体编排 — 互动课堂脚本生成
WriterAgent(导演规划 → 并行分幕) → ReviewerAgent(独立审核 → 带反馈重写)
"""
import asyncio
import json
import logging
import re
import time
from typing import Any, TypedDict, NotRequired

from langgraph.graph import StateGraph, START, END

from backend.src.ai_core.llm_config import llm
from backend.src.utils.prompt_loader import load_prompt, fill_prompt
from backend.src.utils.json_parser import parse_llm_json
from backend.src.service.path.classroom import (
    _classroom_segment_narration_text,
    _normalize_lesson,
    _GENERIC_CLASSROOM_PHRASES,
    _content_evidence_terms,
    generate_classroom_audio,
)

logger = logging.getLogger(__name__)

# -----
# 常量
# -----

# 固定四幕，顺序不可变（与 _fallback_lesson / 前端契约一致）
_SEGMENT_IDS = ("lead-in", "concept", "exercise", "feynman")

# 幕 id → segment.type（前端 isResourceScene/isQuizScene/isFeynmanScene 依赖）
_SEGMENT_TYPES = {
    "lead-in": "hook",
    "concept": "concept",
    "exercise": "exercise",
    "feynman": "feynman",
}

# 审核不通过时只重写被点名的模块，最多进行一轮定向优化。
_MAX_REVIEW_RETRIES = 1

```

```

_REVIEW_PASS_SCORE = 80          # 通过阈值: score >= 80 且结构契约通过
_MAX_SEGMENT_CONCURRENCY = 4     # 四幕并行生成
_REVIEW_FEEDBACK_MAX_LEN = 400   # 回灌给 writer 的总体意见截断, 防止 prompt 膨胀
_TTS_PREWARM_CONCURRENCY = 2
_tts_prewarm_sem = asyncio.Semaphore(_TTS_PREWARM_CONCURRENCY)
_tts_prewarm_tasks: set[asyncio.Task] = set()

# 蓝图缺失时的幕默认 goal / focus (保证任何情况都产出"四幕齐全"蓝图)
_DEFAULT_GOALS = {
    "lead-in": "结合学生画像说明这个知识点与学生的关系",
    "concept": "讲清知识点的定义、结构、关系和使用边界",
    "exercise": "用一道真实练习检查概念、步骤或易错点",
    "feynman": "用自己的话反讲, 并在右侧对话中补齐漏洞",
}

_DEFAULT_FOCUS = {
    "lead-in": "用户专业、年级、目标与该知识点的具体联系",
    "concept": "节点中的核心术语、公式、步骤、例子和易错点",
    "exercise": "节点测验中的题干、选项、答案和解析",
    "feynman": "三句话反讲任务、一个例子、一次右侧对话追问",
}

_SCENE_RESPONSIBILITIES = {
    "lead-in": "结合学生画像或学习目标, 说明该知识点与学生的具体关系。",
    "concept": "讲清知识点的定义、结构、关系、公式、步骤或边界。",
    "exercise": "说明现有练习将检查的知识点、判断依据或易错点。",
    "feynman": "给出具体反讲任务, 要求学生在右侧对话框作答。",
}

# -----
# State
# -----

class ClassroomState(TypedDict):
    # —— 输入 (generate_classroom_lesson 注入) ——
    path_id: int
    node_id: int
    user_id: int
    subject: str
    topic: str
    summary: str
    knowledge_tags: list[str]
    quiz_config: dict[str, Any]
    quiz_snapshot: dict[str, Any]
    resources: list[dict[str, str]]
    portrait_context: str
    fallback_lesson: dict[str, Any] # _fallback_lesson(...) 预计算结果, 异常兜底用
    llm_priority: NotRequired[str] # 默认 "high"
    max_retries: NotRequired[int] # 默认 _MAX_REVIEW_RETRIES
    trace_id: NotRequired[str]    # 一次课堂请求的日志关联标识

```

```

# —— Writer 输出 ——
teaching_outline: NotRequired[dict[str, Any]] # 导演产物；重写轮次复用，不重复规划
raw_lesson: NotRequired[dict[str, Any]] # 4 幕原始拼接（归一化前），Reviewer 审核对象
lesson: NotRequired[dict[str, Any]] # _normalize_lesson 归一化后的最终课堂

# —— Reviewer 输出 ——
review_passed: NotRequired[bool]
review_score: NotRequired[float]
review_feedback: NotRequired[str] # 未通过时的总体意见
review_issues: NotRequired[list[dict[str, Any]]] # [{segment_id, category, message}]
retry_count: NotRequired[int]

# -----
# Writer — 导演规划 + 并行分幕
# -----

async def writer_node(state: ClassroomState) -> dict:
    """首次先规划教学主线，再并行生成四幕；重写轮次复用蓝图并带审核意见重写。"""
    started_at = time.perf_counter()
    path_id = state.get("path_id")
    node_id = state.get("node_id")
    retry_count = state.get("retry_count", 0)
    trace_id = state.get("trace_id", "-")
    logger.info("[ClassroomWriter] 开始生成 trace=%s path=%s node=%s retry=%s", trace_id, path_id, node_id, retry_count)
    try:
        outline = state.get("teaching_outline")
        if outline is None:
            outline = await _plan_teaching_outline(state)
        raw_lesson = await _generate_segments(state, outline)
        lesson = _normalize_lesson(
            raw_lesson,
            state.get("fallback_lesson"),
            topic=state.get("topic", ""),
            summary=state.get("summary", ""),
            knowledge_tags=state.get("knowledge_tags", []),
            resources=state.get("resources", []),
        )
        logger.info(
            "[ClassroomWriter] 生成完成 trace=%s path=%s node=%s retry=%s segments=%s elapsed=%.2fs",
            trace_id,
            path_id,
            node_id,
            retry_count,
            len(raw_lesson.get("segments", [])),
            time.perf_counter() - started_at,
        )
    return {"teaching_outline": outline, "raw_lesson": raw_lesson, "lesson": lesson}

```

```

except Exception:
    logger.exception("[ClassroomWriter] 生成失败 trace=%s path=%s node=%s elapsed=%.2fs", trace_id,
path_id, node_id, time.perf_counter() - started_at)
    return {"lesson": {}, "raw_lesson": {}, "review_passed": False}

async def _plan_teaching_outline(state: ClassroomState) -> dict:
    """导演阶段：1 次 LLM 调用产出四幕蓝图和本节总结。解析失败时用默认蓝图。"""
    started_at = time.perf_counter()
    user_id_int = int(state.get("user_id", 0))
    llm_priority = state.get("llm_priority", "high")
    prompt_text = fill_prompt(
        load_prompt("classroom/writer_planner"),
        subject=state.get("subject", "未知"),
        topic=state.get("topic", ""),
        summary=state.get("summary", ""),
        knowledge_tags=json.dumps(state.get("knowledge_tags", []), ensure_ascii=False),
        quiz_config=json.dumps(state.get("quiz_config", {}), ensure_ascii=False),
        quiz_snapshot=json.dumps(state.get("quiz_snapshot", {}), ensure_ascii=False)[:900],
        resources_json=json.dumps(state.get("resources", []), ensure_ascii=False)[:2200],
        evidence_terms=json.dumps(
            _content_evidence_terms(
                state.get("topic", ""),
                state.get("summary", ""),
                *(state.get("knowledge_tags", []) or []),
                *[
                    value
                    for resource in (state.get("resources", []) or [])
                    if isinstance(resource, dict)
                    for value in (resource.get("title"), resource.get("summary"))
                ],
            ),
            ensure_ascii=False,
        ),
        portrait_context=state.get("portrait_context", "暂无画像数据"),
    )
    raw: dict = {}
    try:
        logger.info("[ClassroomWriter] 蓝图请求模型 trace=%s path=%s node=%s", state.get("trace_id", "-"),
state.get("path_id"), state.get("node_id"))
        response = await llm.ainvoke(prompt_text, priority=llm_priority, user_id=user_id_int, pool="cl
assroom")
        parsed = parse_llm_json(response.content)
        if isinstance(parsed, dict):
            raw = parsed
    except Exception:
        logger.exception("[ClassroomWriter] 蓝图规划失败 trace=%s path=%s node=%s", state.get("trace_id"
, "-"), state.get("path_id"), state.get("node_id"))
        outline = _normalize_outline(raw, state)
        logger.info(

```

```

        "[ClassroomWriter] 蓝图完成 trace=%s path=%s node=%s source=%s elapsed=%.2fs",
        state.get("trace_id", "-"),
        state.get("path_id"),
        state.get("node_id"),
        "llm" if raw else "default",
        time.perf_counter() - started_at,
    )
    return outline

def _normalize_outline(raw: dict, state: ClassroomState) -> dict:
    """按 _SEGMENT_IDS 固定顺序重排/去重 segments, 缺失的幕用默认 goal/focus/style 补齐。"""
    topic = state.get("topic", "当前知识点")
    default_segments = [
        {
            "id": sid,
            "goal": _DEFAULT_GOALS.get(sid, "完成本幕讲解"),
            "focus": _DEFAULT_FOCUS.get(sid, f"围绕 {topic} 展开"),
            "style": "清晰、分步、贴合画像",
        }
        for sid in _SEGMENT_IDS
    ]
    by_id: dict[str, dict] = {}
    segs = raw.get("segments") if isinstance(raw.get("segments"), list) else []
    for item in segs:
        if not isinstance(item, dict):
            continue
        sid = item.get("id")
        if sid not in _SEGMENT_TYPES:
            continue
        by_id[str(sid)] = {
            "goal": str(item.get("goal") or _DEFAULT_GOALS.get(sid, "")),
            "focus": str(item.get("focus") or _DEFAULT_FOCUS.get(sid, f"围绕 {topic} 展开")),
            "style": str(item.get("style") or "清晰、分步、贴合画像"),
        }
    merged = []
    for default in default_segments:
        merged.append(by_id.get(default["id"], default))
    takeaways = raw.get("key_takeaways") if isinstance(raw.get("key_takeaways"), list) else []
    return {
        "title": str(raw.get("title") or topic),
        "personal_summary": str(raw.get("personal_summary") or f"围绕 {topic} 完成一次互动课堂。"),
        "main_thread": str(raw.get("main_thread") or f"先分清{topic}要解决的问题，再拆开核心关系，用资料验证，最后反讲。"),
        "learning_summary": str(raw.get("learning_summary") or f"本节围绕“{topic}”理解核心概念，并用资料核对后完成一次反讲。"),
        "key_takeaways": [str(item) for item in takeaways if str(item).strip()][:4],
        "feynman_plan": raw.get("feynman_plan") if isinstance(raw.get("feynman_plan"), dict) else {},
        "segments": merged,
    }

```



```

async def _generate_segments(state: ClassroomState, outline: dict) -> dict:
    """首轮生成四幕；审核后的定向优化只重写被点名的幕。"""
    started_at = time.perf_counter()
    sem = asyncio.Semaphore(_MAX_SEGMENT_CONCURRENCY)
    issues = state.get("review_issues") or []
    global_feedback = state.get("review_feedback") or ""
    retry_count = state.get("retry_count", 0)
    previous_raw = state.get("raw_lesson") if isinstance(state.get("raw_lesson"), dict) else {}
    previous_by_id = {
        str(segment.get("id")): segment
        for segment in previous_raw.get("segments", [])
        if isinstance(segment, dict) and str(segment.get("id")) in _SEGMENT_IDS
    }
    targeted_ids = {
        str(issue.get("segment_id"))
        for issue in issues
        if isinstance(issue, dict) and str(issue.get("segment_id")) in _SEGMENT_IDS
    }
    is_targeted_retry = retry_count > 0 and len(previous_by_id) == len(_SEGMENT_IDS) and bool(targeted_ids)

    if retry_count > 0 and not is_targeted_retry:
        logger.warning(
            "[ClassroomWriter] 审核未给出可定位模块，不重写整课 path=%s node=%s retry=%s",
            state.get("path_id"), state.get("node_id"), retry_count,
        )
        return previous_raw

    parallel_ids = _SEGMENT_IDS[:-1]
    logger.info(
        "[ClassroomWriter] 并行幕开始 trace=%s path=%s node=%s scenes=%s mode=%s",
        state.get("trace_id", "-"),
        state.get("path_id"), state.get("node_id"), ",".join(parallel_ids),
        "targeted" if is_targeted_retry else "initial",
    )
    parallel_jobs = [
        _gen_segment(state, outline, sid, index, sem, issues, global_feedback)
        for index, sid in enumerate(parallel_ids)
        if not is_targeted_retry or sid in targeted_ids
    ]
    generated_parallel = await asyncio.gather(*parallel_jobs) if parallel_jobs else []
    generated_by_id = {str(segment.get("id")): segment for segment in generated_parallel if isinstance(segment, dict)}
    parallel_segments = [generated_by_id.get(sid) or previous_by_id.get(sid, {})] for sid in parallel_ids]
    logger.info(
        "[ClassroomWriter] 并行幕完成 trace=%s path=%s node=%s valid=%s/%s elapsed=%.2fs",
        state.get("trace_id", "-"),

```

```

        state.get("path_id"),
        state.get("node_id"),
        sum(bool(segment) for segment in parallel_segments),
        len(parallel_ids),
        time.perf_counter() - started_at,
    )
    feynman_started_at = time.perf_counter()
    should_rewrite_feynman = not is_targeted_retry or _SEGMENT_IDS[-1] in targeted_ids
    if should_rewrite_feynman:
        logger.info("[ClassroomWriter] 费曼幕开始 trace=%s path=%s node=%s", state.get("trace_id", "-")
, state.get("path_id"), state.get("node_id"))
        feynman = await _gen_segment(
            state,
            outline,
            _SEGMENT_IDS[-1],
            len(_SEGMENT_IDS) - 1,
            sem,
            issues,
            global_feedback,
            completed_segments=parallel_segments,
        )
    else:
        feynman = previous_by_id.get(_SEGMENT_IDS[-1], {})
        logger.info("[ClassroomWriter] 费曼幕复用 path=%s node=%s", state.get("path_id"), state.get("no
de_id"))
        logger.info(
            "[ClassroomWriter] 费曼幕完成 trace=%s path=%s node=%s valid=%s elapsed=%.2fs total=%.2fs",
            state.get("trace_id", "-"),
            state.get("path_id"),
            state.get("node_id"),
            bool(feynman),
            time.perf_counter() - feynman_started_at,
            time.perf_counter() - started_at,
        )
    raw_segments = parallel_segments + [feynman]
    return {
        "title": outline.get("title", state.get("topic", "")),
        "personal_summary": outline.get("personal_summary", ""),
        "learning_summary": outline.get("learning_summary", ""),
        "key_takeaways": outline.get("key_takeaways", []),
        "segments": raw_segments,
    }

async def _gen_segment(
    state: ClassroomState,
    outline: dict,
    scene_id: str,
    index: int,
    sem: asyncio.Semaphore,

```

```

        issues: list[dict],
        global_feedback: str,
        completed_segments: list[dict] | None = None,
    ) -> dict:
        """生成单幕；费曼幕可接收前三幕的压缩结果。"""
        started_at = time.perf_counter()
        segs = outline.get("segments") or []
        info = segs[index] if index < len(segs) else {}
        feynman_plan = json.dumps(outline.get("feynman_plan") or {}, ensure_ascii=False)
        prompt_text = fill_prompt(
            load_prompt("classroom/writer_segment"),
            subject=state.get("subject", "未知"),
            topic=state.get("topic", ""),
            summary=state.get("summary", ""),
            knowledge_tags=json.dumps(state.get("knowledge_tags", []), ensure_ascii=False),
            portrait_context=state.get("portrait_context", "暂无画像数据"),
            resources_json=json.dumps(state.get("resources", []), ensure_ascii=False)[:1800],
            evidence_terms=json.dumps(
                _content_evidence_terms(
                    state.get("topic", ""),
                    state.get("summary", ""),
                    *(state.get("knowledge_tags", []) or []),
                    *[
                        value
                        for resource in (state.get("resources", []) or [])
                        if isinstance(resource, dict)
                        for value in (resource.get("title"), resource.get("summary"))
                    ],
                ),
            ),
            ensure_ascii=False,
        ),
        scene_id=scene_id,
        scene_index=index + 1,
        scene_total=len(_SEGMENT_IDS),
        scene_goal=info.get("goal", ""),
        scene_focus=info.get("focus", ""),
        scene_style=info.get("style", ""),
        previous_scene_focus=_seg_focus(outline, index - 1),
        next_scene_focus=_seg_focus(outline, index + 1),
        main_thread=outline.get("main_thread", ""),
        feynman_plan=feynman_plan,
        completed_segments=_completed_segment_context(completed_segments),
        review_feedback=_feedback_for_segment(scene_id, issues, global_feedback),
        forbidden_phrases=json.dumps(_GENERIC_CLASSROOM_PHRASES, ensure_ascii=False),
    )
    user_id_int = int(state.get("user_id", 0))
    llm_priority = state.get("llm_priority", "high")
    queue_started_at = time.perf_counter()
    try:
        logger.info("[ClassroomWriter] 单幕等待并发槽 trace=%s path=%s node=%s scene=%s", state.get("tra

```

```

ce_id", "-"), state.get("path_id"), state.get("node_id"), scene_id)
    async with sem:
        logger.info("[ClassroomWriter] 单幕请求模型 trace=%s path=%s node=%s scene=%s queue_wait=%.2fs", state.get("trace_id", "-"), state.get("path_id"), state.get("node_id"), scene_id, time.perf_counter() - queue_started_at)
        response = await llm.ainvoke(prompt_text, priority=llm_priority, user_id=user_id_int, pool="classroom")
        parsed = parse_llm_json(response.content)
        if not isinstance(parsed, dict):
            logger.warning("[ClassroomWriter] 单幕返回非 JSON trace=%s path=%s node=%s scene=%s elapsed=%.2fs", state.get("trace_id", "-"), state.get("path_id"), state.get("node_id"), scene_id, time.perf_counter() - started_at)
            return {}
        parsed["id"] = scene_id
        parsed["type"] = _SEGMENT_TYPES.get(scene_id, scene_id)
        narration_text = _classroom_segment_narration_text(parsed)
        if narration_text:
            task = asyncio.create_task(
                _prewarm_segment_audio(
                    user_id=user_id_int,
                    path_id=int(state.get("path_id", 0)),
                    node_id=int(state.get("node_id", 0)),
                    scene_id=scene_id,
                    text=narration_text,
                )
            )
            _tts_prewarm_tasks.add(task)
            task.add_done_callback(_tts_prewarm_tasks.discard)
        logger.info("[ClassroomWriter] 单幕完成 trace=%s path=%s node=%s scene=%s chars=%s elapsed=%.2fs", state.get("trace_id", "-"), state.get("path_id"), state.get("node_id"), scene_id, len(str(parsed.get("teacher_speech") or parsed.get("script") or "")), time.perf_counter() - started_at)
        return parsed
    except Exception:
        logger.exception("[ClassroomWriter] 单幕生成失败 trace=%s path=%s node=%s scene=%s elapsed=%.2fs", state.get("trace_id", "-"), state.get("path_id"), state.get("node_id"), scene_id, time.perf_counter() - started_at)
        return {}

async def _prewarm_segment_audio(
    user_id: int,
    path_id: int,
    node_id: int,
    scene_id: str,
    text: str,
) -> None:
    """分幕生成后立即预热语音；失败不影响课堂正文和审核。"""
    try:
        async with _tts_prewarm_sem:
            await generate_classroom_audio(text, user_id)

```

```

        logger.info(
            "[ClassroomTTS] 预热完成 path=%s node=%s scene=%s chars=%s",
            path_id,
            node_id,
            scene_id,
            len(text),
        )
    except Exception:
        logger.exception(
            "[ClassroomTTS] 预热失败 path=%s node=%s scene=%s",
            path_id,
            node_id,
            scene_id,
        )

def _completed_segment_context(segments: list[dict] | None) -> str:
    """只把前三幕的关键内容交给费曼幕，避免把完整 JSON 塞进 prompt。"""
    if not segments:
        return "前三幕尚未完成；当前不是费曼幕时忽略此字段。"
    compact = []
    for segment in segments:
        if not isinstance(segment, dict):
            continue
        compact.append({
            "id": segment.get("id", ""),
            "title": segment.get("title", ""),
            "teacher_speech": segment.get("teacher_speech") or segment.get("script", ""),
            "points": (segment.get("points") or segment.get("board_items") or [])[:4],
            "example": segment.get("example", ""),
            "question": (segment.get("question") or {}).get("prompt", ""),
        })
    return json.dumps(compact, ensure_ascii=False)[:5000] if compact else "前三幕没有返回可用内容。"

def _seg_focus(outline: dict, index: int) -> str:
    segs = outline.get("segments") or []
    if 0 <= index < len(segs):
        return str(segs[index].get("focus") or "")
    return ""

def _feedback_for_segment(segment_id: str, issues: list[dict], global_feedback: str) -> str:
    """把 issues 按 segment_id 过滤（None 视为全局）拼成该幕专属反馈，总体意见附加为末尾。"""
    parts: list[str] = []
    for issue in issues:
        if not isinstance(issue, dict):
            continue
        sid = issue.get("segment_id")
        if sid is None or str(sid) == segment_id:

```

```

        msg = str(issue.get("message") or "")
        if msg:
            parts.append(msg)
    if global_feedback:
        parts.append(f"[总体意见] {global_feedback}")
    return "\n".join(parts) if parts else ""

# -----
# Reviewer — 独立审核（对象是归一化前的 raw_lesson）
# -----

async def reviewer_node(state: ClassroomState) -> dict:
    """四个审核智能体并行检查分幕；只把问题回灌给对应模块。"""
    started_at = time.perf_counter()
    trace_id = state.get("trace_id", "-")
    raw_lesson = state.get("raw_lesson") or state.get("lesson")
    if not raw_lesson or not isinstance(raw_lesson.get("segments"), list):
        logger.warning("[ClassroomReviewer] 审核跳过：课堂为空 trace=%s path=%s node=%s", trace_id, state.get("path_id"), state.get("node_id"))
        return {"review_passed": False, "review_score": 0, "review_feedback": "课堂结构为空", "review_issues": [], "retry_count": state.get("retry_count", 0)}

    segments_by_id = {
        str(segment.get("id")): segment
        for segment in raw_lesson.get("segments", [])
        if isinstance(segment, dict) and str(segment.get("id")) in _SEGMENT_IDS
    }
    missing_ids = [scene_id for scene_id in _SEGMENT_IDS if scene_id not in segments_by_id]
    if missing_ids:
        issues = [{"segment_id": scene_id, "category": "structure", "message": "缺少课堂模块"} for scene_id in missing_ids]
        retry_count = state.get("retry_count", 0)
        can_retry = retry_count < state.get("max_retries", _MAX_REVIEW_RETRIES)
        logger.warning(
            "[ClassroomReviewer] 结构审核失败 trace=%s path=%s node=%s missing=%s retry=%s can_retry=%s",
            trace_id,
            state.get("path_id"),
            state.get("node_id"),
            ",".join(missing_ids),
            retry_count,
            can_retry,
        )
    return {
        "review_passed": False,
        "review_score": 0,
        "review_feedback": "课堂模块不完整",
        "review_issues": issues if can_retry else [],
        "retry_count": retry_count + 1 if can_retry else retry_count,
    }

```

```

    }

    structural_issues: list[dict[str, Any]] = []
    for scene_id, segment in segments_by_id.items():
        script = str(segment.get("teacher_speech") or segment.get("script") or "").strip()
        board_items = segment.get("board_items") if isinstance(segment.get("board_items"), list) else []

        points = segment.get("points") if isinstance(segment.get("points"), list) else []
        question = segment.get("question") if isinstance(segment.get("question"), dict) else {}
        if not str(segment.get("title") or "").strip() or len(script) < 30 or len(board_items) < 2 or
len(points) < 2 or len(str(question.get("prompt") or "").strip()) < 6:
            structural_issues.append({"segment_id": scene_id, "category": "structure", "message": "讲
稿、要点或互动问题不完整"})
        concept_issue = _concept_definition_contract_issue(
            segments_by_id.get("concept", {}),
            state.get("knowledge_tags", []),
        )
    if concept_issue:
        structural_issues.append(concept_issue)
    if structural_issues:
        retry_count = state.get("retry_count", 0)
        can_retry = retry_count < state.get("max_retries", _MAX_REVIEW_RETRIES)
        logger.warning(
            "[ClassroomReviewer] 展示契约失败 trace=%s path=%s node=%s scenes=%s retry=%s can_retry=%s",
            trace_id,
            state.get("path_id"),
            state.get("node_id"),
            ",".join(str(item.get("segment_id")) for item in structural_issues),
            retry_count,
            can_retry,
        )
    return {
        "review_passed": False,
        "review_score": 0,
        "review_feedback": "课堂结构不完整",
        "review_issues": structural_issues if can_retry else [],
        "retry_count": retry_count + 1 if can_retry else retry_count,
    }

    user_id_int = int(state.get("user_id", 0))
    llm_priority = state.get("llm_priority", "high")
    logger.info("[ClassroomReviewer] 并行审核开始 trace=%s path=%s node=%s scenes=%s", trace_id, state.g
et("path_id"), state.get("node_id"), ",".join(_SEGMENT_IDS))

    async def review_scene(scene_id: str) -> tuple[str, bool, float, str, list[dict]]:
        review_started_at = time.perf_counter()
        prompt_text = fill_prompt(
            load_prompt("classroom/reviewer_segment"),
            subject=state.get("subject", "未知"),
            topic=state.get("topic", ""),

```

```

summary=state.get("summary", ""),
knowledge_tags=json.dumps(state.get("knowledge_tags", []), ensure_ascii=False),
resources_json=json.dumps(state.get("resources", []), ensure_ascii=False)[:1800],
evidence_terms=json.dumps(
    _content_evidence_terms(
        state.get("topic", ""),
        state.get("summary", ""),
        *(state.get("knowledge_tags", []) or []),
    ),
    ensure_ascii=False,
),
scene_id=scene_id,
scene_responsibility=_SCENE_RESPONSIBILITIES[scene_id],
segment_json=json.dumps(segments_by_id[scene_id], ensure_ascii=False)[:2600],
)
try:
    logger.info("[ClassroomReviewer] 单幕请求模型 trace=%s path=%s node=%s scene=%s", trace_id,
state.get("path_id"), state.get("node_id"), scene_id)
    response = await llm.ainvoke(prompt_text, priority=llm_priority, user_id=user_id_int, pool
="classroom")
    result = parse_llm_json(response.content)
    if not isinstance(result, dict):
        result = {}
    raw_passed, score, feedback, issues = _parse_review_result(result)
    normalized_issues = [
        {"segment_id": scene_id, "category": str(issue.get("category") or "quality"), "message
": str(issue.get("message") or feedback or "需要优化当前模块")}
        for issue in issues if isinstance(issue, dict)
    ]
    passed = raw_passed and score >= _REVIEW_PASS_SCORE
    if not passed and not normalized_issues:
        normalized_issues = [{"segment_id": scene_id, "category": "quality", "message": feedba
ck or "当前模块未达到审核要求"}]
    logger.info(
        "[ClassroomReviewer] 单幕审核完成 trace=%s path=%s node=%s scene=%s passed=%s score=%.1f
issues=%s elapsed=%.2fs",
        trace_id,
        state.get("path_id"),
        state.get("node_id"),
        scene_id,
        passed,
        score,
        len(normalized_issues),
        time.perf_counter() - review_started_at,
    )
    return scene_id, passed, score, feedback, normalized_issues
except Exception:
    logger.exception("[ClassroomReviewer] 分幕审核失败, 保留当前幕 trace=%s path=%s node=%s scene
=%s elapsed=%.2fs", trace_id, state.get("path_id"), state.get("node_id"), scene_id, time.perf_counter(
) - review_started_at)

```



```

        return scene_id, True, 0.0, "", []

    scene_reviews = await asyncio.gather(*(review_scene(scene_id) for scene_id in _SEGMENT_IDS))
    passed = all(item[1] for item in scene_reviews)
    score = min((item[2] for item in scene_reviews), default=0.0)
    feedback = "\n".join(item[3] for item in scene_reviews if item[3])(:_REVIEW_FEEDBACK_MAX_LEN)
    issues = [issue for item in scene_reviews for issue in item[4]]
    retry_count = state.get("retry_count", 0)
    actionable_issues = [
        issue for issue in issues
        if isinstance(issue, dict) and str(issue.get("segment_id")) in _SEGMENT_IDS
    ]
    can_retry = not passed and retry_count < state.get("max_retries", _MAX_REVIEW_RETRIES) and bool(actionable_issues)
    next_retry_count = retry_count + 1 if can_retry else retry_count
    logger.info(
        "[ClassroomReviewer] 审核完成 trace=%s path=%s node=%s passed=%s score=%.1f issues=%s targeted=%s retry=%s elapsed=%.2fs",
        trace_id,
        state.get("path_id"),
        state.get("node_id"),
        passed,
        score,
        len(issues),
        ", ".join(str(item.get("segment_id")) for item in actionable_issues) or "none",
        next_retry_count,
        time.perf_counter() - started_at,
    )
    return {
        # 没有精确模块的问题时不允许全课重写, 保留当前完整课堂。
        "review_passed": passed or not can_retry,
        "review_score": score,
        "review_feedback": feedback[:_REVIEW_FEEDBACK_MAX_LEN] if not passed else "",
        "review_issues": actionable_issues if can_retry else [],
        "retry_count": next_retry_count,
    }

def _parse_review_result(result: dict) -> tuple[bool, float, str, list[dict]]:
    """容错解析审核结果: passed 兼容 bool/str, score 取 float, issues 非 list 则置 []。"""
    passed = result.get("passed", False)
    if isinstance(passed, str):
        passed = passed.lower() in ("true", "yes", "1", "是", "pass", "通过")
    try:
        score = float(result.get("score", 0))
    except (TypeError, ValueError):
        score = 0.0
    feedback = str(result.get("feedback") or "")
    issues = result.get("issues")
    if not isinstance(issues, list):

```

```

issues = []
return bool(passed), score, feedback, issues

def _concept_definition_contract_issue(segment: dict, knowledge_tags: list[Any]) -> dict | None:
    """拦截把例子、结论或数值当作概念定义的核心讲解。"""
    if not isinstance(segment, dict):
        return {"segment_id": "concept", "category": "concept_definition", "message": "核心概念模块缺失"}

    speech = str(segment.get("teacher_speech") or segment.get("script") or "").strip()
    opening = speech[:190]
    definition_pattern = re.compile(
        r"(?:^[。；；，，])^[。；；，，]{0,30}?(?:是(?:一种|指|用来|用于)|指的是|用于|表示的是)"
    )
    definition_count = len(definition_pattern.findall(opening))
    if definition_count < 1:
        return {
            "segment_id": "concept",
            "category": "concept_definition",
            "message": "核心概念必须先用“X 是一种…/X 指…/X 用于…”明确下定义，不能直接从例子、数值或结果开始。",
        }

    targets: list[str] = []
    for tag in knowledge_tags:
        if isinstance(knowledge_tags, list) else []:
            term = str(tag or "").strip()
            if 2 <= len(term) <= 32 and term not in targets:
                targets.append(term)
    required_definitions = 2 if len(targets) >= 2 else 1
    if definition_count < required_definitions:
        return {
            "segment_id": "concept",
            "category": "concept_definition",
            "message": "核心概念包含多个术语时，需分别定义至少两个术语，再讲它们的关系和例子。",
        }
    return None

# -----
# Router / Graph
# -----

def should_continue(state: ClassroomState) -> str:
    if state.get("review_passed"):
        logger.info("[ClassroomGraph] 审核通过或无需全课重写，结束 trace=%s path=%s node=%s", state.get("trace_id", "-"), state.get("path_id"), state.get("node_id"))
        return "end"
    issues = state.get("review_issues") or []
    if issues:

```

```

        logger.info("[ClassroomGraph] 定向优化模块 trace=%s path=%s node=%s scenes=%s", state.get("trace_id", "-"), state.get("path_id"), state.get("node_id"), ",".join(str(item.get("segment_id")) for item in issues))
        return "writer"
        logger.warning("[ClassroomGraph] 审核未通过但无可修复模块, 结束 trace=%s path=%s node=%s", state.get("trace_id", "-"), state.get("path_id"), state.get("node_id"))
        return "end"

def build_classroom_graph():
    workflow = StateGraph(ClassroomState)
    workflow.add_node("writer", writer_node)
    workflow.add_node("reviewer", reviewer_node)
    workflow.add_edge(START, "writer")
    workflow.add_edge("writer", "reviewer")
    workflow.add_conditional_edges(
        "reviewer",
        should_continue,
        {"writer": "writer", "end": END},
    )
    return workflow.compile()

classroom_graph = build_classroom_graph()
"""
LangGraph 多智能体编排 — 个性化学习路径生成
LeaderAgent(规划大纲) → ExecutorAgent(并行分组生成节点) → ReviewerAgent(审核)
"""
import asyncio
import json
import logging
import time
from typing import TypedDict, NotRequired

from langgraph.graph import StateGraph, START, END

from backend.src.ai_core.llm_config import llm
from backend.src.utils.prompt_loader import load_prompt, fill_prompt
from backend.src.utils.json_parser import parse_llm_json

logger = logging.getLogger(__name__)

# 每组最多生成的节点数（并行分组）
_GROUP_SIZE = 4
_MAX_GROUP_CONCURRENCY = 2
_GROUP_RETRY_ATTEMPTS = 2

_COGNITIVE_LEVELS = ["记忆", "理解", "应用", "分析", "评价", "创造"]

```

```

def _safe_int(value, default: int = 0) -> int:
    try:
        return int(value)
    except (TypeError, ValueError):
        return default

def _normalize_topic_outline(raw, subject: str, node_count: int = 0) -> list[dict]:
    if not isinstance(raw, list):
        return []

    normalized: list[dict] = []
    seen: set[str] = set()
    for index, item in enumerate(raw, 1):
        if not isinstance(item, dict):
            continue
        topic = str(item.get("topic") or item.get("title") or "").strip()
        if not topic or topic in seen:
            continue
        seen.add(topic)
        level = str(item.get("cognitive_level") or _COGNITIVE_LEVELS[min(index - 1, len(_COGNITIVE_LEVELS) - 1)])
        normalized.append({
            "topic": topic,
            "module": str(item.get("module") or item.get("category") or subject).strip(),
            "cognitive_level": level,
            "learning_goal": str(item.get("learning_goal") or f"理解并掌握{topic}").strip(),
            "prerequisite_topics": item.get("prerequisite_topics") if isinstance(item.get("prerequisite_topics"), list) else [],
            "key_points": item.get("key_points") if isinstance(item.get("key_points"), list) else [topic],
            "micro_example": str(item.get("micro_example") or f"完成一个关于{topic}的小练习").strip(),
        })
        if node_count and len(normalized) >= node_count:
            break
    return normalized

def _fallback_topic_outline(subject: str, node_count: int = 0) -> list[dict]:
    target = max(8, min(18, _safe_int(node_count, 12) or 12))
    templates = [
        ("学习目标与知识地图", "概念奠基"),
        ("核心概念与基本术语", "概念奠基"),
        ("基础结构与表示方法", "概念奠基"),
        ("基本规则与操作流程", "方法操作"),
        ("典型方法一：基础应用", "方法操作"),
        ("典型方法二：组合应用", "方法操作"),
        ("关键例题与解题步骤", "案例应用"),
        ("常见错误与辨析", "误区辨析"),
    ]

```

```

        ("小型综合任务", "综合迁移"),
        ("阶段复盘与知识迁移", "综合迁移"),
        ("进阶应用场景", "案例应用"),
        ("最终综合练习", "综合迁移"),
    ]
    outline = []
    for index in range(target):
        name, module = templates[index % len(templates)]
        topic = f"{subject}: {name}"
        outline.append({
            "topic": topic,
            "module": module,
            "cognitive_level": _COGNITIVE_LEVELS[min(index, len(_COGNITIVE_LEVELS) - 1)],
            "learning_goal": f"围绕「{subject}」掌握{name}, 并能完成对应检查任务",
            "prerequisite_topics": [outline[-1]["topic"]] if outline else [],
            "key_points": [subject, name, module],
            "micro_example": f"用一个小例子检查「{name}」是否掌握",
        })
    return outline

def _fallback_group_nodes(group: list[dict], group_start: int) -> list[dict]:
    nodes: list[dict] = []
    for index, item in enumerate(group):
        order_index = group_start + index
        topic = str(item.get("topic") or f"学习节点 {order_index}").strip()
        key_points = item.get("key_points") if isinstance(item.get("key_points"), list) else [topic]
        nodes.append({
            "topic": topic,
            "order_index": order_index,
            "knowledge_tags": key_points[:5],
            "prerequisites": [group_start + index - 1] if order_index > 1 else [],
            "resource_types": ["document", "ppt", "mindmap"],
            "quiz_config": {"count": 5, "threshold": 0.7},
            "description": str(item.get("learning_goal") or f"掌握{topic}的核心概念、典型应用和常见误区").s
trip(),
        })
    return nodes

async def parse_or_repair_leader_result(raw_text: str, state: dict, *, retry_llm: bool = True) -> dict
:
    """Parse Path Leader JSON; repair once, then fall back to a stable outline."""
    subject = str(state.get("subject") or "通用学习")
    requested_count = _safe_int(state.get("node_count"), 0)
    difficulty = str(state.get("difficulty") or "medium")
    user_id_int = _safe_int(state.get("user_id"), 0)
    llm_priority = state.get("llm_priority", "high")

    try:

```

```

        result = parse_llm_json(raw_text)
        if not isinstance(result, dict):
            result = {}
    except Exception as parse_error:
        logger.warning("[PathLeader] JSON 解析失败, 尝试修复: %s", parse_error)
        result = {}
        if retry_llm:
            repair_prompt = (
                "你是 JSON 修复器。请只修复下面内容为合法 JSON, 不要新增解释文字, 不要使用 markdown。\\n"
                "要求: 第一个字符必须是 {, 最后一个字符必须是 }; 保留原有 topic_outline 结构; 无法确定的字段用合
理短文本补齐。\\n\\n"
                f"原始内容: \\n{str(raw_text or '')[:6000]}\\n"
            )
            try:
                repaired = await llm.ainvoke(repair_prompt, priority=llm_priority, user_id=user_id_int
, pool="path")
                parsed = parse_llm_json(repaired.content)
                if isinstance(parsed, dict):
                    result = parsed
            except Exception:
                logger.exception("[PathLeader] JSON 修复失败, 使用本地兜底大纲")

    topic_outline = _normalize_topic_outline(result.get("topic_outline"), subject, requested_count)
    if not topic_outline:
        logger.warning("[PathLeader] topic_outline 为空, 使用本地兜底大纲 subject=%s", subject)
        topic_outline = _fallback_topic_outline(subject, requested_count)

    node_count = _safe_int(result.get("node_count"), len(topic_outline)) or len(topic_outline)
    node_count = max(1, min(node_count, len(topic_outline)))
    return {
        "topic_outline": topic_outline[:node_count],
        "node_count": node_count,
        "difficulty": str(result.get("difficulty") or difficulty),
    }

# -----
# State
# -----

class PathState(TypedDict):
    user_id: str
    subject: str
    difficulty: str
    node_count: int
    portrait_context: str
    mastery_context: str
    kb_context: str
    learning_guidance: str
    # Leader 输出

```

```

    if (/^(^|\.|.)bing\.com$/i.test(url.hostname) && url.pathname.startsWith('/search') && query.length >
80) {
        return false
    }
    return true
} catch {
    return false
}
}

/**
 * v-html 渲染后的富文本点击守卫：拦截 javascript:/data:/bing 长链接等异常链接。
 * 在包裹 v-html 的容器上绑定 @click.capture="handleRenderedMarkdownClick"。
 */
export const handleRenderedMarkdownClick = event => {
    const anchor = event.target?.closest?('(a')
    if (!anchor) return
    if (isSafeRenderedHref(anchor.getAttribute('href') || '')) return
    event.preventDefault()
    event.stopPropagation()
    console.warn('[Markdown] 已拦截异常富文本链接:', anchor.getAttribute('href'))
}

const isImageResourceUrl = url => {
    return /\. (png|jpe?g|webp|gif|bmp|svg)(?:[?#].*)?$/i.test(String(url || ''))
}

const renderInlineMarkdown = (value) => {
    let text = escapeHtml(value)

    text = text.replace(/`([^\`]+)`/g, '<code>${1}</code>')
    text = text.replace(/\*(\[^\*]+\)*\*/g, '<strong>${1}</strong>')
    text = text.replace(/_([^\_]+)_/g, '<strong>${1}</strong>')
    text = text.replace(/\*([^\*\n]+)\*/g, '<em>${1}</em>')
    text = text.replace(/_([^\_\n]+)_/g, '<em>${1}</em>')
    text = text.replace(/!\[([^\]]*)\]\(((?:https?:\/\/|\/|mailto:|\/)[^\s])+\))/g, (_, label, url) => {
        const href = escapeHtml(resolveMarkdownHref(url))
        if (!href) return label
        return `<a class="md-image-link" href="${href}" target="_blank" rel="noopener noreferrer"></a>`
    })
    text = text.replace(/!\[([^\]]*)\]\(((?:https?:\/\/|\/|mailto:|\/)[^\s])+\))/g, (_, label, url) => {
        const href = escapeHtml(resolveMarkdownHref(url))
        if (!href) return label

        if (!/^mailto:/i.test(url) && isImageResourceUrl(url)) {
            return `<a class="md-image-link" href="${href}" target="_blank" rel="noopener noreferrer"></a>`
        }
    }
}

```

```

    return `<a href="${href}" target="_blank" rel="noopener noreferrer">${label}</a>`
  })

  return text
}

const isTableSeparator = (line) => {
  return /\s*\|?\s*:-{3,}?:?\s*(\\|\\s*:-{3,}?:?\s*)+\\|?\s*$/ .test(line)
}

const renderTable = (tableLines) => {
  const rows = tableLines.map(line => {
    return line
      .trim()
      .replace(/^\\|/, '')
      .replace(/\\|$/, '')
      .split('|')
      .map(cell => renderInlineMarkdown(cell.trim()))
  })

  const header = rows[0] || []
  const body = rows.slice(2)

  return `
    <div class="md-table-wrap">
      <table>
        <thead><tr>${header.map(cell => `<th>${cell}</th>`).join('')}</tr></thead>
        <tbody>${body.map(row => `<tr>${row.map(cell => `<td>${cell}</td>`).join('')}</tr>`).join('')}
      </tbody>
    </table>
  </div>
  `
}

/**
 * 轻量 Markdown → HTML (含 LaTeX 公式, 复用 renderMath)。
 * 行为与聊天页 ChatView 完全一致; 公式/列表/表格/代码块/引用/链接均可渲染。
 * 使用处需配 .markdown-body 样式与 @click.capture="handleRenderedMarkdownClick" 守卫。
 */
export function renderMarkdown(content) {
  const text = String(content || '').trim()
  if (!text) return ''

  const lines = text.split(/\\r?\\n/)
  const html = []
  let paragraph = []
  let listItems = []
  let listType = ''
  let codeLines = []
  let inCodeBlock = false

```



```

let codeLanguage = ''

const flushParagraph = () => {
  if (!paragraph.length) return
  html.push('<p>${paragraph.map(renderInlineMarkdown).join('<br>')}</p>')
  paragraph = []
}

const flushList = () => {
  if (!listItems.length) return
  const tag = listType === 'ol' ? 'ol' : 'ul'
  html.push('<${tag}>${listItems.map(item => `<li>${renderInlineMarkdown(item)}</li>`).join('')}</${tag}>')
  listItems = []
  listType = ''
}

const flushCode = () => {
  const languageLabel = codeLanguage ? `<span>${escapeHtml(codeLanguage)}</span>` : ''
  html.push('<div class="md-code-block">${languageLabel}<pre><code>${escapeHtml(codeLines.join('\n'))}</code></pre></div>')
  codeLines = []
  codeLanguage = ''
}

for (let index = 0; index < lines.length; index += 1) {
  const rawLine = lines[index]
  const line = rawLine.trim()

  if (line.startsWith('```')) {
    if (inCodeBlock) {
      flushCode()
      inCodeBlock = false
    } else {
      flushParagraph()
      flushList()
      inCodeBlock = true
      codeLanguage = line.replace(/^```/, '').trim()
    }
    continue
  }

  if (inCodeBlock) {
    codeLines.push(rawLine)
    continue
  }

  if (!line) {
    flushParagraph()
    flushList()
  }
}

```

```
        continue
    }

    if (line.includes('|') && lines[index + 1] && isTableSeparator(lines[index + 1])) {
        flushParagraph()
        flushList()
        const tableLines = [rawLine, lines[index + 1]]
        index += 2
        while (index < lines.length && lines[index].includes('|') && lines[index].trim()) {
            tableLines.push(lines[index])
            index += 1
        }
        index -= 1
        html.push(renderTable(tableLines))
        continue
    }

    const headingMatch = line.match(/^(#{1,4})\s+(.+)$/ )
    if (headingMatch) {
        flushParagraph()
        flushList()
        const level = Math.min(headingMatch[1].length + 2, 6)
        html.push(`<h${level}>${renderInlineMarkdown(headingMatch[2])}</h${level}>`)
        continue
    }

    const blockquoteMatch = line.match(/^>\s?(.+)$/ )
    if (blockquoteMatch) {
        flushParagraph()
        flushList()
        html.push(`<blockquote>${renderInlineMarkdown(blockquoteMatch[1])}</blockquote>`)
        continue
    }

    const orderedMatch = line.match(/^\\d+\\.\\s+(.+)$/ )
    const unorderedMatch = line.match(/^[-*]\\s+(.+)$/ )

    if (orderedMatch || unorderedMatch) {
        flushParagraph()
        const currentType = orderedMatch ? 'ol' : 'ul'

        if (listType && listType !== currentType) {
            flushList()
        }

        listType = currentType
        listItems.push((orderedMatch?.[1] || unorderedMatch?.[1] || '').trim())
        continue
    }
}
```

```
        flushList()
        paragraph.push(rawLine)
    }

    if (inCodeBlock) {
        flushCode()
    }
    flushParagraph()
    flushList()

    return renderMath(html.join(''))
}
import { mergeSpeakerNotes, splitSpeakerNotes, stripSpeakerNotes } from './pptSpeakerNotes'

export const PPTIST_PAYLOAD_KEY = 'zhiban:pptist:payload'

const WIDTH = 1000
const HEIGHT = 562.5

const PALETTES = {
    default: ['#163f8f', '#5f8fc3', '#c9dce9', 'ffffff'],
    minimal: ['#111216', '#3b3f4a', '#6b6f7a', 'ffffff'],
    dark: ['#7aa2f7', '#bb9af7', '#7dcfff', '#1a1b26'],
    warm: ['#d94860', '#e36a2d', '#f2a341', 'fff7ef'],
    green: ['#11695f', '#28b487', '#a7f3d0', 'f6fffb']
}

const escapeHtml = value => String(value || '')
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#39;')

const cleanText = value => String(value || '')
    .replace(/<!--[\s\S]*?-->/g, ' ')
    .replace(/<\/?[^\>\n]+>/g, ' ')
    .replace(/^#{1,6}\s+/gm, '')
    .replace(/^\[-*+·\]\s+/gm, '')
    .replace(/\*\*(.*?)\*/g, '$1')
    .replace(/\n{3,}/g, '\n\n')
    .trim()

const splitBlocks = value => cleanText(value)
    .split(/\r?\n/)
    .map(line => line.trim())
    .filter(Boolean)
    .slice(0, 8)

const estimateLines = (value, charsPerLine = 33) => Math.max(1, Math.ceil(String(value || '').length /
```

```

    charsPerLine))

const paginateBlocks = (blocks, {
  maxLines = 12,
  maxItems = 5,
  charsPerLine = 33
} = {}) => {
  const pages = []
  let current = []
  let lineCount = 0

  for (const block of blocks) {
    const text = cleanText(block)
    if (!text) continue

    const blockLines = estimateLines(text, charsPerLine)
    const shouldStartNewPage = current.length && (
      lineCount + blockLines > maxLines ||
      current.length >= maxItems
    )

    if (shouldStartNewPage) {
      pages.push(current)
      current = []
      lineCount = 0
    }

    current.push(text)
    lineCount += blockLines
  }

  if (current.length) pages.push(current)
  return pages.length ? pages : [[]]
}

const htmlParagraph = (value, { size = 24, color = '#333333', bold = false, align = 'left' } = {}) => {
  const tagOpen = bold ? '<strong>' : ''
  const tagClose = bold ? '</strong>' : ''
  return `<p style="text-align:${align};">${tagOpen}<span style="font-size:${size}px;color:${color};">
  ${escapeHtml(value)}</span>${tagClose}</p>`
}

const htmlList = (items, { size = 22, color = '#333333' } = {}) => {
  if (!items.length) return htmlParagraph('', { size, color })
  return `<ul style="font-size:${size}px;color:${color};">${items.map(item => `<li><p>${escapeHtml(item)}</p></li>`).join('')}</ul>`
}

const textElement = (id, content, left, top, width, height, options = {}) => ({

```

```

    type: 'text',
    id,
    left,
    top,
    width,
    height,
    rotate: 0,
    content,
    defaultFontName: options.font || '',
    defaultColor: options.color || '#333333',
    lineHeight: options.lineHeight || 1.25,
    fixedHeight: Boolean(options.fixedHeight),
    vAlign: options.vAlign || 'top',
    ...(options.fill ? { fill: options.fill } : {}),
    ...(options.textType ? { textType: options.textType } : {}),
    ...(options.inset ? { inset: options.inset } : {})
  })
})

const rectElement = (id, left, top, width, height, fill, options = {}) => ({
  type: 'shape',
  id,
  left,
  top,
  width,
  height,
  viewBox: [200, 200],
  path: 'M 0 0 L 200 0 L 200 200 L 0 200 Z',
  fill,
  fixedRatio: false,
  rotate: 0,
  ...(options.opacity ? { opacity: options.opacity } : {}),
  ...(options.radius ? { radius: options.radius } : {})
})

const paletteForTheme = themeId => {
  const id = String(themeId || '').toLowerCase()
  if (id.includes('dark') || id.includes('night') || id.includes('mocha') || id.includes('terminal'))
    return PALETTES.dark
  if (id.includes('warm') || id.includes('sunset') || id.includes('retro')) return PALETTES.warm
  if (id.includes('green') || id.includes('science')) return PALETTES.green
  if (id.includes('minimal') || id.includes('white')) return PALETTES.minimal
  return PALETTES.default
}

const normalizeSlide = slide => {
  const title = cleanText(slide?.title || slide?.heading || 'Untitled')
  const bodyParts = splitSpeakerNotes(slide?.text || slide?.content || slide?.body || '')
  const blockNotes = (Array.isArray(slide?.blocks) ? slide.blocks : [])
    .map(block => splitSpeakerNotes(block?.text || block?.content || block).notes)
    .filter(Boolean)

```

```

    const blocks = Array.isArray(slide?.blocks) && slide.blocks.length
      ? slide.blocks.map(block => cleanText(stripSpeakerNotes(block?.text || block?.content || block))).
        filter(Boolean)
      : splitBlocks(bodyParts.body)
    const notes = cleanText(mergeSpeakerNotes(slide?.notes, slide?.speaker_notes, bodyParts.notes, ...blockNotes))
    return { title, blocks, notes }
  }

const buildCoverSlide = (slide, index, palette) => {
  const [primary, secondary, accent, paper] = palette
  const id = `zhiban_slide_${index + 1}`
  return {
    id,
    background: { type: 'solid', color: paper },
    remark: slide.notes || '',
    elements: [
      rectElement(`${id}_band`, 0, 0, 1000, 562.5, primary, { opacity: 0.08 }),
      rectElement(`${id}_left`, 0, 0, 18, 562.5, primary),
      rectElement(`${id}_accent`, 72, 365, 250, 8, accent),
      textElement(`${id}_title`, htmlParagraph(slide.title, { size: 46, color: primary, bold: true })),
      72, 145, 820, 150, {
        color: primary,
        textType: 'title',
        fixedHeight: true,
        vAlign: 'middle'
      },
    ],
    textElement(`${id}_sub`, htmlParagraph(slide.blocks.slice(0, 2).join(' '), { size: 20, color: secondary })),
    76, 315, 760, 92, {
      color: secondary,
      textType: 'subtitle'
    },
  ])
}

const buildContentSlide = (slide, index, palette) => {
  const [primary, secondary, accent, paper] = palette
  const id = `zhiban_slide_${index + 1}`
  const blocks = slide.blocks.length ? slide.blocks : ['No content']
  const estimatedLines = blocks.reduce((sum, block) => sum + estimateLines(block), 0)
  const dense = estimatedLines > 10 || blocks.length > 4
  const bodyFontSize = estimatedLines > 13 ? 17 : estimatedLines > 10 ? 18 : blocks.length > 5 ? 19 :
22
  const bodyWidth = dense ? 820 : 600
  return {
    id,
    background: { type: 'solid', color: paper },
    remark: slide.notes || '',
    elements: [

```

```

    rectElement(`${id}_top`, 0, 0, 1000, 72, primary, { opacity: 0.92 }),
    rectElement(`${id}_accent`, 64, 104, 68, 6, accent),
    textElement(`${id}_title`, htmlParagraph(slide.title, { size: 30, color: '#ffffff', bold: true }
), 58, 14, 850, 52, {
    color: '#ffffff',
    textType: 'title',
    fixedHeight: true,
    vAlign: 'middle'
}),
    textElement(`${id}_body`, htmlList(blocks, { size: bodyFontSize, color: '#1f2937' }), 74, 124, bodyWidth, 390, {
    color: '#1f2937',
    textType: 'content',
    lineHeight: dense ? 1.18 : 1.3,
    fixedHeight: true
}),
    ...(dense ? [
    rectElement(`${id}_visual`, 720, 135, 208, 248, secondary, { opacity: 0.14 }),
    rectElement(`${id}_visual_bar`, 720, 135, 208, 14, secondary),
    textElement(`${id}_visual_text`, htmlParagraph(slide.title, { size: 20, color: primary, bold:
true, align: 'center' }), 742, 220, 164, 82, {
    color: primary,
    fixedHeight: true,
    vAlign: 'middle'
}),
    ] : [])
  ]
}
}

```

```

const expandSlidesForPptist = slides => {
  const expanded = []

  slides.forEach((slide, slideIndex) => {
    if (slideIndex === 0) {
      expanded.push(slide)
      return
    }

    const pages = paginateBlocks(slide.blocks, {
      maxLines: 12,
      maxItems: 5,
      charsPerLine: 34
    })

    pages.forEach((blocks, pageIndex) => {
      expanded.push({
        ...slide,
        title: pageIndex ? `${slide.title} (cont.)` : slide.title,
        blocks
      })
    })
  })
}

```

```

    })
  })
})

return expanded
}

export const toPptistPayload = ({ title = 'Zhiban PPT', slides = [], themeId = '' } = {}) => {
  const palette = paletteForTheme(themeId)
  const normalizedSlides = (Array.isArray(slides) ? slides : [])
    .map(normalizeSlide)
    .filter(slide => slide.title || slide.blocks.length)
  const pptistSlides = expandSlidesForPptist(normalizedSlides)

  return {
    title,
    viewportSize: WIDTH,
    viewportRatio: HEIGHT / WIDTH,
    theme: {
      themeColors: palette.slice(0, 3).concat(['#ffc000', '#4472c4', '#70ad47']),
      fontColor: '#333333',
      fontName: '',
      backgroundColor: palette[3],
      shadow: { h: 3, v: 3, blur: 2, color: '#808080' },
      outline: { width: 2, color: palette[0], style: 'solid' }
    },
    slides: pptistSlides.map((slide, index) => (
      index === 0
        ? buildCoverSlide(slide, index, palette)
        : buildContentSlide(slide, index, palette)
    ))
  }
}

export const openPptistEditor = payload => {
  const key = `${PPTIST_PAYLOAD_KEY}:${Date.now()}`
  localStorage.setItem(key, JSON.stringify(payload))
  window.open(`/pptist/index.html?source=${encodeURIComponent(key)}`, '_blank', 'noopener,noreferrer')
}

const NOTE_LABEL_RE = /^(?:\u8bb2\u7a3f|\u5907\u6ce8|\u6f14\u8bb2\u7a3f|speaker\s*notes?|speaker_notes?|notes?)\s*[:\uff1a]\s*(.*)$/i

const unwrapLine = line => String(line || '')
  .replace(/^s*?:[-*\u2022]\s*)?/, '')
  .replace(/^s*>s?/, '')
  .trim()

const matchNoteLine = line => unwrapLine(line).match(NOTE_LABEL_RE)

export const splitSpeakerNotes = value => {

```



```
const lines = String(value || '').split(/\r?\n/)
const body = []
const notes = []
let readingQuotedNote = false

for (const rawLine of lines) {
  const matched = matchNoteLine(rawLine)
  if (matched) {
    const noteText = matched[1]?.trim()
    if (noteText) notes.push(noteText)
    readingQuotedNote = /^s*>/.test(rawLine)
    continue
  }

  if (readingQuotedNote && /^s*>/.test(rawLine)) {
    const noteText = unwrapLine(rawLine)
    if (noteText) notes.push(noteText)
    continue
  }

  readingQuotedNote = false
  body.push(rawLine)
}

return {
  body: body.join('\n').trim(),
  notes: notes.join('\n').trim()
}
}

export const stripSpeakerNotes = value => splitSpeakerNotes(value).body

export const mergeSpeakerNotes = (...values) => values
  .map(value => String(value || '').trim())
  .filter(Boolean)
  .join('\n')

const SESSION_KEY = 'zhiban_quiz_sessions'
// v3: 题面必须与后端会话记录一致, 旧版本地缓存可能保存了重排前的选项。
export const QUIZ_SCHEMA_VERSION = 3

const uid = prefix => `${prefix}-${Date.now()}-${Math.random().toString(16).slice(2)}`

const readSessions = () => {
  try {
    const data = JSON.parse(localStorage.getItem(SESSION_KEY) || '[]')
    return Array.isArray(data) ? data : []
  } catch {
    return []
  }
}
```

```

const writeSessions = sessions => {
  localStorage.setItem(SESSION_KEY, JSON.stringify(sessions))
}

const stripFence = value =>
  String(value || '')
    .trim()
    .replace(/^```(?:json|markdown|md)?\s*/i, '')
    .replace(/``$/i, '')
    .trim()

const normalizeAnswer = value => {
  if (typeof value === 'boolean') return value ? 'A' : 'B'
  if (Array.isArray(value)) return value.map(k => String(k || '').trim().toUpperCase()).filter(Boolean)
    .sort().join(',')
  const text = String(value ?? '').trim()
  // 处理 JSON 序列化后的布尔和字母
  if (/^(?:true|false)$/i.test(text)) return text.toUpperCase() === 'TRUE' ? 'A' : 'B'
  try {
    const parsed = JSON.parse(text)
    if (typeof parsed === 'boolean') return parsed ? 'A' : 'B'
    if (Array.isArray(parsed)) return parsed.map(k => String(k || '').trim().toUpperCase()).filter(Boolean)
      .sort().join(',')
  } catch {}
  return text
    .replace(/^(答案|正确答案|参考答案)[: ]?\s*/i, '')
    .replace(/^( ( )?\s*([A-F])\s* )\.\. ]?\s*$/i, '$1')
    .toUpperCase()
}

const normalizeQuestion = (item, index) => {
  const rawOptions = item.options || item.choices || item.option || []
  const options = Array.isArray(rawOptions)
    ? rawOptions.map((option, optionIndex) => {
      const text = typeof option === 'string'
        ? option
        : String(option.text || option.content || option.value || '')
      const match = text.match(/^\s*([A-F])(\.\. ]?\s*(.*)$/i)

      return {
        key: String(option.key || option.label || match?.[1] || String.fromCharCode(65 + optionIndex))
          .toUpperCase(),
        text: match?.[2] || text
      }
    })
    : []

  // 只用字符串类型的值作为题干, 跳过 object (如 record 包装格式 {question: {...}})
  const stem = (() => {

```

```

    for (const key of ['stem', 'question', 'content', 'title', 'question_text']) {
      const val = item[key]
      if (typeof val === 'string') return val
    }
    return `第 ${index + 1} 题`
  })()

const questionType = String(item.question_type || item.type || '').toLowerCase()
const isMulti =
  item.multi ||
  item.is_multi ||
  item.multiple ||
  ['multiple', 'multi', 'checkbox', 'multi_choice'].includes(questionType)

const rawAnswer = item.answer ?? item.correctAnswer ?? item.correct_answer ?? item.correct ?? ''
const answerStr = String(rawAnswer || '').trim()
const hasMultiple = /[,\s]/.test(answerStr) || /^[A-F]{2,}$/i.test(answerStr)

const multi = isMulti || hasMultiple

return {
  question_id: item.question_id || item.questionId || item.id || null,
  id: item.question_id || item.questionId || item.id || uid(`question-${index + 1}`),
  type: questionType || (multi ? 'multiple' : (options.length ? 'choice' : 'short')),
  question_type: questionType || (multi ? 'multi_choice' : (options.length ? 'single_choice' : 'fill_blank')),
  stem: String(stem).trim(),
  options,
  answer: multi
    ? answerStr.toUpperCase().replace(/^[A-F,]/g, '').split(/[,,\s]+/).filter(Boolean).sort().join(
      ',')
    : normalizeAnswer(answerStr),
  multi,
  explanation: String(item.explanation || item.analysis || item.reason || '').trim()
}
}

const parseJsonQuestions = content => {
  const text = stripFence(content)
  const candidates = [text]
  const arrayMatch = text.match(/\[\[\\s\\S]*\\]/)
  const objectMatch = text.match(/\{\{\\s\\S]*\\}/)

  if (arrayMatch) candidates.push(arrayMatch[0])
  if (objectMatch) candidates.push(objectMatch[0])

  for (const candidate of candidates) {
    try {
      const parsed = JSON.parse(candidate)
      const list = Array.isArray(parsed) ? parsed : parsed.questions || parsed.items || parsed.data ||

```

```

    []

    if (Array.isArray(list) && list.length) {
      // 解包 record 包装格式 {record_id, question: {...}} → {...}
      const unwrapped = list.map(item =>
        item.question && typeof item.question === 'object' ? item.question : item
      )
      return unwrapped.map(normalizeQuestion).filter(item => item.stem)
    }
  } catch {
    // Try next candidate.
  }
}

return []
}

const parseMarkdownQuestions = content => {
  const text = stripFence(content)
  const blocks = text
    .split(/\n(?:=|s*(?:#{1,4}\s*)?(?:第\s*)?\d+\s*[\.\. ) ])|\n-{3,}\n/)
    .map(block => block.trim())
    .filter(Boolean)

  const sourceBlocks = blocks.length > 1
    ? blocks
    : text.split(/\n{2,}/).map(block => block.trim()).filter(Boolean)

  return sourceBlocks.map((block, index) => {
    const lines = block.split(/\r?\n/).map(line => line.trim()).filter(Boolean)
    const stemLines = []
    const options = []
    let answer = ''
    let explanation = ''

    lines.forEach(line => {
      const optionMatch = line.match(/^(?:[-*]\s*)?([A-D])(\.\. )\s*(.+)$/i)
      const answerMatch = line.match(/^(?:[-*]\s*)?(?:答案|正确答案|参考答案|answer)[:: ]\s*(.+)$/i)
      const explanationMatch = line.match(/^(?:[-*]\s*)?(?:解析|解释|说明|analysis)[:: ]\s*(.+)$/i)

      if (optionMatch) {
        options.push({ key: optionMatch[1].toUpperCase(), text: optionMatch[2].trim() })
        return
      }

      if (answerMatch) {
        answer = answerMatch[1].trim()
        return
      }

      if (explanationMatch) {

```

```

        explanation = explanationMatch[1].trim()
        return
    }

    stemLines.push(line.replace(/^#{1,4}s*/, '').replace(/^(?:第\s*)?\d+\s*[\.\, \)]\s*/, ''))
})

return normalizeQuestion({ stem: stemLines.join('\n'), options, answer, explanation }, index)
}).filter(item => item.stem && (item.options.length || item.answer))
}

export const parseQuizQuestions = content => {
    const jsonQuestions = parseJsonQuestions(content)
    if (jsonQuestions.length) return jsonQuestions
    return parseMarkdownQuestions(content)
}

export const looksLikeQuizContent = content => {
    const text = String(content || '')
    if (!text.trim()) return false
    if (parseQuizQuestions(text).length > 0) return true
    return /question_type|single_choice|multi_choice|true_false|正确答案|参考答案|答案[: ]|解析[: ]/i.test
    (text)
}

export const QUIZ_BANK_KEY = SESSION_KEY
export const QUIZ_ATTEMPTS_KEY = 'zhiban_quiz_attempts'

export const upsertQuizSet = payload => {
    const questions = payload.questions || parseQuizQuestions(payload.content)
    if (!questions.length) return null

    const sessionId =
        payload.id ||
        (payload.sourceId ? `quiz-resource-${payload.sourceId}` : uid('quiz'))

    const session = {
        schemaVersion: QUIZ_SCHEMA_VERSION,
        id: sessionId,
        sourceId: payload.sourceId || '',
        sessionId: payload.sessionId || payload.session_id || '',
        title: payload.title || 'AI 生成题目',
        content: payload.content || '',
        fileType: payload.fileType || 'exercise',
        filename: payload.filename || '',
        questionCount: questions.length,
        questions,
        attempts: payload.attempts || [],
        bestScore: payload.bestScore ?? null,
        lastScore: payload.lastScore ?? null,
    }
}

```

```

        lastAttemptAt: payload.lastAttemptAt || '',
        createdAt: payload.createdAt || new Date().toISOString()
    }

    const sessions = readSessions()
    const next = [session, ...sessions.filter(item => item.id !== sessionId && item.sourceId !== session
    .sourceId)]
    writeSessions(next)
    window.dispatchEvent(new CustomEvent('zhiban-quiz-bank-updated', { detail: session }))
    return session
}

export const readQuizBank = () => readSessions()

export const getQuizSet = quizId => {
    const session = readSessions().find(item => item.id === quizId) || null
    // 清理已缓存的老格式垃圾数据 (record 包装 → stem 成了 [object Object])
    if (session?.questions?.length) {
        const clean = session.questions.filter(q => q.stem && q.stem !== '[object Object]')
        if (clean.length !== session.questions.length) {
            const sessions = readSessions()
            if (clean.length === 0) {
                // 全部损坏, 移除整个 session, 让上游重新生成
                const next = sessions.filter(s => s.id !== session.id)
                writeSessions(next)
                return null
            }
            session.questions = clean
            session.questionCount = clean.length
            const next = sessions.map(s => s.id === session.id ? session : s)
            writeSessions(next)
        }
    }
    return session
}

const readAttempts = () => {
    try {
        const data = JSON.parse(localStorage.getItem(QUIZ_ATTEMPTS_KEY) || '[]')
        return Array.isArray(data) ? data : []
    } catch {
        return []
    }
}

const writeAttempts = attempts => {
    localStorage.setItem(QUIZ_ATTEMPTS_KEY, JSON.stringify(attempts))
}

export const readQuizAttempts = quizId => {

```

```
    const attempts = readAttempts()
    return quizId ? attempts.filter(item => item.quizId === quizId) : attempts
  }

export const recordQuizAttempt = payload => {
  const quizId = String(payload.quizId || '')
  if (!quizId) return null

  const attempt = {
    id: uid('attempt'),
    quizId,
    sessionId: payload.sessionId || '',
    title: payload.title || '',
    score: Number(payload.score || 0),
    total: Number(payload.total || 0),
    percent: Number(payload.percent || 0),
    answers: payload.answers || {},
    results: payload.results || {},
    createdAt: new Date().toISOString()
  }

  const attempts = [attempt, ...readAttempts()]
  writeAttempts(attempts)

  const sessions = readSessions()
  const next = sessions.map(session => {
    if (session.id !== quizId) return session
    const sessionAttempts = [attempt, ...(session.attempts || [])]
    const bestScore = Math.max(Number(session.bestScore || 0), attempt.percent)
    return {
      ...session,
      attempts: sessionAttempts,
      bestScore,
      lastScore: attempt.percent,
      lastAttemptAt: attempt.createdAt
    }
  })
  writeSessions(next)
  window.dispatchEvent(new CustomEvent('zhiban-quiz-bank-updated', { detail: getQuizSet(quizId) }))
  return attempt
}

import katex from 'katex'

const render = (formula, displayMode) => {
  try {
    return katex.renderToString(formula.trim(), {
      displayMode,
      throwOnError: false,
      trust: true,
    })
  }
}
```

```

    } catch {
      return formula
    }
  }
}

/**
 * 修复 LLM 常见公式问题：
 * 1. 裸  $\begin{...}\end{...}$  没包 $ → 补  $$$$...$ 
 * 2. $ 放反:  $A$$  →  $$$A$$ 
 */
function fixBrokenLatex(text) {
  // 1. 清理  $\begin{env}...\end{env}$  周围游离的 $, 统一用 $$ 包裹
  // LLM 常见错误: 裸奔、$ 放一边、$...$ 内嵌块级环境
  text = text.replace(
    /\$*\begin\{([a-zA-Z*]+\s\S)*?\end\{\1\}\$*/g,
    (match) => `$$$${match.replace(/\$+|\$+$/g, '')}}$$`,
  )
  // 2. $ 放反: 数学表达式后紧跟 $ 但前面无 $ 配对 → 补开头的 $
  return text.replace(
    /(^[^$\{\}_\^_])([A-Za-z0-9×÷±≤≥≠≈·ΣΠ∫√∞∂∇+\\-*=<>+])\$(?=[\s,。、;:,.;:( )、—\x])/g
    ,
    '$1$$$2$',
  )
}

/** CJK 及中文标点 — 这些字符不应出现在纯数学公式中 */
const NON_MATH_RE = /[一-龠𠂇-𠄎𠄎-𠄎 𠄎-𠄎-𠄎]/
const CJK_STRIP_RE = /[一-龠𠂇-𠄎𠄎-𠄎 𠄎-𠄎-𠄎,。、;:;! ? () [] 《》 "" '']/g

/** 标点/空白 — 不需要 karaoke 高亮, 也不消耗 timedWords */
const PUNCT_RE = /^[\s,。、;:;! ? ,. ; ! ? ]+$/

/**
 * 将文本渲染为带 karaoke 逐词高亮的 HTML。
 * 与 renderMath 一样处理 LaTeX, 同时将普通文字按 timedWords 状态包入 karaoke span。
 *
 * @param {string} text - 待渲染文本
 * @param {Array<{text: string, isDone: boolean, isCurrent: boolean}>} timedWords - 全 slide 的词状态
 * @param {number} wordOffset - 当前 item 在 timedWords 中的起始下标
 * @returns {{ html: string, consumed: number }}
 */
export function renderKaraokeHTML(text, timedWords, wordOffset = 0) {
  if (!text || typeof text !== 'string') return { html: text, consumed: 0 }

  const fixed = fixBrokenLatex(text)
  const words = timedWords || []
  let wi = wordOffset
  let consumedTotal = 0
  let result = ''

```



```

const parts = fixed.split(/(\\$\\s\\$)*?\\$\\|\\\\[\\s\\$]*?\\\\|\\$[^\\s\\$][^\\$]*?\\$\\\\\\\\(\\s\\$)*?\\\\\\\\)/g
)

for (const part of parts) {
  if (!part) continue

  // LaTeX 公式 — 直接渲染, 不消耗 timedWords 位置
  if (/^\\$\\|\\\\[\\$|^\\\\(\\.test(part)) {
    if (part.startsWith('$') || part.startsWith('\\(')) {
      const f = part.replace(/^\\$\\|\\$\\$/g, '').replace(/\\\\\\\\[\\\\\\\\]$/g, '')
      result += render(f, true)
    } else {
      const f = part.replace(/^\\$\\|\\$\\$/g, '').replace(/\\\\\\\\(\\\\\\\\)$/g, '')
      if (NON_MATH_RE.test(f)) {
        const stripped = f.replace(CJK_STRIP_RE, '').trim()
        if (stripped && stripped.length > 1) {
          try { result += render(stripped, false) } catch { result += `${f}` }
        } else {
          result += `${f}`
        }
      } else {
        result += render(f, false)
      }
    }
  }
  continue
}

// 逐字拆分 — timedWords 已由上层插值为字级, 每个 Unicode 字符对应一条
const chars = [...part]
for (const ch of chars) {
  if (PUNCT_RE.test(ch)) {
    result += ch
    continue
  }
  const tw = wi < words.length ? words[wi] : null
  // timedWords 不足时, 宁可保持未高亮, 也不要后面的字误判成已完成
  // 这样可以避免“段尾文字在一开始就直接亮起来”的错位现象。
  const isDone = tw ? tw.isDone : false
  const isCurrent = tw ? tw.isCurrent : false
  const cls = `karaoke-word${isDone ? ' is-done' : ''}${isCurrent ? ' is-current' : ''}`
  result += `<span class="${cls}">${ch}</span>`
  wi++
  consumedTotal++
}
}

return { html: result, consumed: consumedTotal }
}

/**

```

```

* 将文本中的 LaTeX 公式渲染为 HTML。
* 支持: $$...$$ / \[...\] (块级), $...$ / \(...\) (行内)
*/
export function renderMath(text) {
  if (!text || typeof text !== 'string') return text

  const fixed = fixBrokenLatex(text)

  // 按顺序处理: 块级优先, 避免 $ 干扰 $$
  return fixed
    .replace(/\$\$([\s\S]*?)\$\$/g, (_, f) => render(f, true))
    .replace(/\\[([\s\S]*?)\]\]/g, (_, f) => render(f, true))
    .replace(/\$([\s\S]*?)\$/g, (_, f) => {
      if (NON_MATH_RE.test(f)) {
        // 尝试剥离中文后渲染数学部分
        const stripped = f.replace(CJK_STRIP_RE, '').trim()
        if (stripped && stripped.length > 1) {
          try {
            return render(stripped, false)
          } catch {
            return `$$${f}$$`
          }
        }
      }
      return `$$${f}$$` // 纯中文, 不渲染
    })
    .replace(/\\[([\s\S]*?)\]\]/g, (_, f) => render(f, false))
}

import { resolveApiUrl } from '../api/apis'

const palettes = [
  ['#2f5d9f', '#58aeea', '#e9f7fb'],
  ['#1e8f8c', '#55c9a7', '#e9fbf6'],
  ['#8f5bbf', '#d9a7ff', '#f8f0ff'],
  ['#c96f22', '#f2b705', '#fff7df'],
  ['#d94d7b', '#ef8fb0', '#fff0f5'],
  ['#566bd8', '#8eb1ff', '#eff4ff']
]

const documentMotifs = [
  { icon: 'notes', side: '#f2b705' },
  { icon: 'chart', side: '#55c9a7' },
  { icon: 'stack', side: '#d9a7ff' },
  { icon: 'quote', side: '#ef8fb0' }
]

const typeLabels = {
  document: '文档',
  reference: '文档',

```

```

    ppt: 'PPT',
    powerpoint: 'PPT',
    presentation: 'PPT',
    mindmap: '导图',
    exercise: '题库',
    quiz: '题库',
    image: '图片',
    video: '视频'
  }

  const escapeXml = value => String(value || '')
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')

  const hashText = value => {
    const text = String(value || '')
    let hash = 0
    for (let index = 0; index < text.length; index += 1) {
      hash = ((hash << 5) - hash + text.charCodeAt(index)) | 0
    }
    return Math.abs(hash)
  }

  const compactTitle = value => {
    const title = String(value || '学习资源').replace(/\s+/g, ' ').trim()
    return title.length > 18 ? `${title.slice(0, 18)}...` : title
  }

  const splitTitleLines = value => {
    const title = String(value || '学习资源').replace(/\s+/g, ' ').trim()
    if (title.length <= 10) return [title]
    return [title.slice(0, 10), title.slice(10, 20)]
  }

  const resourceKind = resource => {
    const text = String(`${resource?.type || ''} ${resource?.category || ''} ${resource?.categoryLabel || ''} ${resource?.title || ''}`).toLowerCase()
    if (/image|图片/.test(text)) return 'image'
    if (/video|mp4|视频/.test(text)) return 'video'
    if (/mind|xmind|导图/.test(text)) return 'mindmap'
    if (/ppt|powerpoint|presentation|slide/.test(text)) return 'ppt'
    if (/exercise|quiz|question|exam|题|练习/.test(text)) return 'exercise'
    return 'document'
  }

  const toDataSvg = svg => `data:image/svg+xml;charset=UTF-8,${encodeURIComponent(svg.trim())}`

  const renderPptCover = ({ primary, accent, rawTitle }) => {

```

```

const [lineOne, lineTwo = ''] = splitTitleLines(rawTitle).map(escapeXml)
return toDataSvg(`
  <svg xmlns="http://www.w3.org/2000/svg" width="520" height="280" viewBox="0 0 520 280">
    <defs>
      <linearGradient id="pptBg" x1="0" x2="1" y1="0" y2="1">
        <stop offset="0%" stop-color="${primary}" />
        <stop offset="56%" stop-color="${accent}" />
        <stop offset="100%" stop-color="#ffffff" />
      </linearGradient>
      <filter id="shadow" x="-20%" y="-20%" width="140%" height="140%">
        <feDropShadow dx="0" dy="10" stdDeviation="10" flood-color="#163f8f" flood-opacity="0.18"/>
      </filter>
    </defs>
    <rect width="520" height="280" rx="20" fill="url(#pptBg)" />
    <circle cx="452" cy="42" r="92" fill="#ffffff" opacity="0.14" />
    <circle cx="64" cy="246" r="116" fill="#ffffff" opacity="0.12" />
    <rect x="54" y="42" width="354" height="196" rx="12" fill="#ffffff" filter="url(#shadow)" />
    <rect x="54" y="42" width="354" height="48" rx="12" fill="${primary}" />
    <rect x="54" y="78" width="354" height="12" fill="${primary}" />
    <text x="78" y="72" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="20" font-weight="900" fill="#ffffff">PPT</text>
    <text x="78" y="136" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="28" font-weight="900" fill="#17345f">${lineOne}</text>
    <text x="78" y="172" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="24" font-weight="800" fill="#31557f">${lineTwo}</text>
    <rect x="78" y="196" width="142" height="9" rx="4.5" fill="${accent}" opacity="0.62" />
    <rect x="78" y="216" width="238" height="9" rx="4.5" fill="${primary}" opacity="0.18" />
    <g transform="translate(424 76)">
      <rect x="0" y="0" width="58" height="40" rx="6" fill="#ffffff" opacity="0.9" />
      <rect x="9" y="9" width="28" height="5" rx="2.5" fill="${primary}" opacity="0.65" />
      <rect x="9" y="20" width="40" height="5" rx="2.5" fill="${accent}" opacity="0.55" />
      <rect x="-18" y="58" width="58" height="40" rx="6" fill="#ffffff" opacity="0.74" />
      <rect x="-9" y="67" width="32" height="5" rx="2.5" fill="${primary}" opacity="0.5" />
      <rect x="-9" y="78" width="42" height="5" rx="2.5" fill="${accent}" opacity="0.48" />
    </g>
  </svg>
`)
}

const renderDocumentCover = ({ primary, accent, paper, rawTitle, resource }) => {
  const [lineOne, lineTwo = ''] = splitTitleLines(rawTitle).map(escapeXml)
  const motif = documentMotifs[hashText(rawTitle || resource?.doc_id) % documentMotifs.length]
  const motifSvg = {
    notes: `
      <rect x="0" y="0" width="86" height="12" rx="6" fill="${primary}" opacity="0.24" />
      <rect x="0" y="28" width="118" height="12" rx="6" fill="${accent}" opacity="0.3" />
      <rect x="0" y="56" width="72" height="12" rx="6" fill="${primary}" opacity="0.18" />
    `,
    chart: `
      <rect x="0" y="38" width="18" height="42" rx="6" fill="${primary}" opacity="0.74" />
    `
  }
}

```

```

    <rect x="30" y="14" width="18" height="66" rx="6" fill="{accent}" opacity="0.74"/>
    <rect x="60" y="28" width="18" height="52" rx="6" fill="{motif.side}" opacity="0.82"/>
    <path d="M0 16L30 2130 16 34-12" fill="none" stroke="{primary}" stroke-width="6" stroke-linecap
="round" opacity="0.32"/>
    ,
    stack: `
    <rect x="0" y="12" width="118" height="76" rx="12" fill="{primary}" opacity="0.16"/>
    <rect x="14" y="0" width="118" height="76" rx="12" fill="{accent}" opacity="0.24"/>
    <rect x="28" y="-12" width="118" height="76" rx="12" fill="ffffff" opacity="0.76"/>
    <rect x="44" y="12" width="70" height="8" rx="4" fill="{primary}" opacity="0.26"/>
    <rect x="44" y="32" width="92" height="8" rx="4" fill="{accent}" opacity="0.32"/>
    ,
    quote: `
    <text x="0" y="70" font-family="Georgia, serif" font-size="92" font-weight="900" fill="{primary
}" opacity="0.18"> "</text>
    <rect x="48" y="18" width="88" height="10" rx="5" fill="{accent}" opacity="0.34"/>
    <rect x="48" y="42" width="118" height="10" rx="5" fill="{primary}" opacity="0.2"/>
    <rect x="48" y="66" width="68" height="10" rx="5" fill="{motif.side}" opacity="0.42"/>
    ,
  }[motif.icon]

return toDataSvg(`
  <svg xmlns="http://www.w3.org/2000/svg" width="520" height="280" viewBox="0 0 520 280">
    <defs>
      <linearGradient id="docBg" x1="0" x2="1" y1="0" y2="1">
        <stop offset="0%" stop-color="{paper}"/>
        <stop offset="46%" stop-color="ffffff"/>
        <stop offset="100%" stop-color="#eef6ff"/>
      </linearGradient>
      <linearGradient id="docBand" x1="0" x2="1">
        <stop offset="0%" stop-color="{primary}"/>
        <stop offset="52%" stop-color="{accent}"/>
        <stop offset="100%" stop-color="{motif.side}"/>
      </linearGradient>
      <filter id="docShadow" x="-20%" y="-20%" width="140%" height="140%">
        <feDropShadow dx="0" dy="12" stdDeviation="12" flood-color="#17345f" flood-opacity="0.16"/>
      </filter>
    </defs>
    <rect width="520" height="280" rx="24" fill="url(#docBg)"/>
    <path d="M0 0h520v80H0z" fill="url(#docBand)"/>
    <path d="M342 0h178v280H342z" fill="{primary}" opacity="0.06"/>
    <path d="M390 0c62 30 96 78 102 144 5 60-17 101-66 124h94V0z" fill="{accent}" opacity="0.12"/>
    <rect x="42" y="38" width="304" height="204" rx="18" fill="ffffff" filter="url(#docShadow)"/>
    <rect x="42" y="38" width="304" height="44" rx="18" fill="#f7fbff"/>
    <path d="M42 64h304" stroke="{primary}" stroke-opacity="0.1" stroke-width="2"/>
    <rect x="66" y="58" width="76" height="10" rx="5" fill="{primary}" opacity="0.18"/>
    <rect x="264" y="56" width="42" height="14" rx="7" fill="{motif.side}" opacity="0.28"/>
    <text x="66" y="124" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="26" font-
weight="900" fill="#17345f">${lineOne}</text>
    <text x="66" y="158" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="22" font-

```

```

weight="800" fill="#4c6382">${lineTwo}</text>
  <rect x="66" y="184" width="218" height="8" rx="4" fill="${primary}" opacity="0.16"/>
  <rect x="66" y="206" width="176" height="8" rx="4" fill="${accent}" opacity="0.24"/>
  <rect x="66" y="224" width="238" height="8" rx="4" fill="${primary}" opacity="0.1"/>
  <g transform="translate(374 126)">
    <rect x="-18" y="-24" width="118" height="104" rx="20" fill="ffffff" opacity="0.78"/>
    ${motifSvg}
  </g>
  <rect x="380" y="34" width="84" height="28" rx="14" fill="ffffff" opacity="0.92"/>
  <text x="422" y="53" text-anchor="middle" font-family="Microsoft YaHei, PingFang SC, sans-serif"
font-size="15" font-weight="900" fill="${primary}">文档</text>
  <path d="M42 38h18v20H42z" fill="${motif.side}" opacity="0.82"/>
</svg>
`)
}

const renderGenericCover = ({ primary, accent, paper, kind, rawTitle }) => {
  const title = escapeXml(compactTitle(rawTitle))
  const label = escapeXml(typeLabels[kind] || '资源')

  return toDataSvg(`
    <svg xmlns="http://www.w3.org/2000/svg" width="520" height="280" viewBox="0 0 520 280">
      <defs>
        <linearGradient id="bg" x1="0" x2="1" y1="0" y2="1">
          <stop offset="0%" stop-color="${paper}" />
          <stop offset="100%" stop-color="ffffff" />
        </linearGradient>
        <linearGradient id="band" x1="0" x2="1">
          <stop offset="0%" stop-color="${primary}" />
          <stop offset="100%" stop-color="${accent}" />
        </linearGradient>
      </defs>
      <rect width="520" height="280" rx="28" fill="url(#bg)" />
      <path d="M0 0h520v86H0z" fill="url(#band)" opacity="0.95" />
      <circle cx="440" cy="56" r="74" fill="ffff" opacity="0.16" />
      <circle cx="488" cy="26" r="36" fill="ffff" opacity="0.14" />
      <rect x="34" y="36" width="92" height="34" rx="17" fill="ffff" opacity="0.9" />
      <text x="80" y="59" text-anchor="middle" font-family="Microsoft YaHei, PingFang SC, sans-serif"
font-size="18" font-weight="800" fill="${primary}">${label}</text>
      <text x="34" y="150" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="32" font-
weight="900" fill="#163f8f">${title}</text>
      <rect x="34" y="188" width="260" height="10" rx="5" fill="${primary}" opacity="0.18" />
      <rect x="34" y="212" width="360" height="10" rx="5" fill="${primary}" opacity="0.12" />
      <rect x="34" y="236" width="190" height="10" rx="5" fill="${primary}" opacity="0.12" />
      <path d="M398 162c34 0 62 28 62 62h-62z" fill="${accent}" opacity="0.55" />
      <path d="M398 224h62c0 34-28 62 62z" fill="${primary}" opacity="0.82" transform="translate(0
-62)" />
    </svg>
  `)
}

```

```

const renderIllustratedResourceCover = ({ primary, accent, paper, kind, rawTitle, resource }) => {
  const [lineOne, lineTwo = ''] = splitTitleLines(rawTitle).map(escapeXml)
  const seed = hashText(rawTitle || resource?.doc_id || resource?.sourceId || kind)
  const warm = ['#f2b705', '#ef8fb0', '#55c9a7', '#8eb1ff'][seed % 4]
  const cool = ['#e9fbf6', '#eff4ff', '#fff7df', '#f8f0ff'][seed % 4]
  const accentTwo = ['#1e8f8c', '#566bd8', '#c96f22', '#8f5bbf'][seed % 4]
  const motif = {
    ppt: `
      <g transform="translate(318 66)">
        <rect x="24" y="20" width="132" height="92" rx="18" fill="#ffffff" opacity="0.7"/>
        <rect x="0" y="0" width="132" height="92" rx="18" fill="#ffffff" filter="url(#shadow)"/>
        <path d="M18 60c26-34 44-18 58-36 20 20 34 40 38 54H18z" fill="${accent}" opacity="0.3"/>
        <circle cx="94" cy="28" r="12" fill="${warm}" opacity="0.86"/>
        <rect x="18" y="18" width="52" height="8" rx="4" fill="${primary}" opacity="0.22"/>
        <rect x="18" y="36" width="76" height="7" rx="3.5" fill="${accentTwo}" opacity="0.24"/>
      </g>
    `,
    mindmap: `
      <g transform="translate(326 58)" fill="none" stroke-linecap="round">
        <path d="M72 76L26 34M72 76L76-36M72 76L-18 76M72 76L88 76" stroke="${primary}" stroke-width="
8" opacity="0.2"/>
        <circle cx="72" cy="76" r="28" fill="#ffffff" stroke="${primary}" stroke-width="0" filter="url
(#shadow)"/>
        <circle cx="26" cy="34" r="18" fill="${accent}" opacity="0.8"/>
        <circle cx="148" cy="40" r="22" fill="${warm}" opacity="0.86"/>
        <circle cx="54" cy="152" r="20" fill="${accentTwo}" opacity="0.78"/>
        <circle cx="160" cy="152" r="18" fill="#ffffff" opacity="0.82"/>
      </g>
    `,
    exercise: `
      <g transform="translate(330 62)">
        <rect x="10" y="0" width="132" height="160" rx="24" fill="#ffffff" filter="url(#shadow)"/>
        <path d="M34 46L16 34-38" fill="none" stroke="${accentTwo}" stroke-width="10" stroke-lineca
p="round" stroke-linejoin="round"/>
        <rect x="34" y="86" width="84" height="10" rx="5" fill="${primary}" opacity="0.18"/>
        <rect x="34" y="112" width="64" height="10" rx="5" fill="${warm}" opacity="0.42"/>
        <circle cx="122" cy="34" r="32" fill="${accent}" opacity="0.16"/>
      </g>
    `,
    document: `
      <g transform="translate(322 54)">
        <rect x="22" y="20" width="118" height="154" rx="22" fill="${primary}" opacity="0.08"/>
        <rect x="0" y="0" width="124" height="160" rx="22" fill="#ffffff" filter="url(#shadow)"/>
        <path d="M92 0v42h32" fill="${cool}"/>
        <path d="M92 0v42h32" fill="none" stroke="${primary}" stroke-opacity="0.1" stroke-width="2"/>
        <rect x="24" y="48" width="58" height="9" rx="4.5" fill="${primary}" opacity="0.18"/>
        <rect x="24" y="74" width="80" height="9" rx="4.5" fill="${accent}" opacity="0.26"/>
        <rect x="24" y="100" width="66" height="9" rx="4.5" fill="${warm}" opacity="0.35"/>
      </g>
    `
  }
}

```

```

    ,
    video: `
      <g transform="translate(330 64)">
        <rect x="0" y="0" width="132" height="96" rx="16" fill="#ffffff" filter="url(#shadow)"/>
        <polygon points="52,28 52,68 92,48" fill="${warm}" opacity="0.86"/>
        <rect x="0" y="106" width="132" height="8" rx="4" fill="${primary}" opacity="0.18"/>
        <rect x="0" y="124" width="76" height="8" rx="4" fill="${accent}" opacity="0.32"/>
        <rect x="84" y="124" width="48" height="8" rx="4" fill="${warm}" opacity="0.22"/>
      </g>
    `
  }[kind] || `
    <g transform="translate(326 62)">
      <rect x="0" y="0" width="148" height="148" rx="34" fill="#ffffff" filter="url(#shadow)"/>
      <path d="M34 112c18-54 48-84 96-96" fill="none" stroke="${accent}" stroke-width="16" stroke-line
cap="round" opacity="0.52"/>
      <circle cx="48" cy="48" r="20" fill="${warm}" opacity="0.86"/>
      <circle cx="104" cy="92" r="28" fill="${primary}" opacity="0.16"/>
    </g>
  `

  return toDataSvg(`
    <svg xmlns="http://www.w3.org/2000/svg" width="520" height="280" viewBox="0 0 520 280">
      <defs>
        <linearGradient id="learningBg" x1="0" x2="1" y1="0" y2="1">
          <stop offset="0%" stop-color="${paper}"/>
          <stop offset="52%" stop-color="#ffffff"/>
          <stop offset="100%" stop-color="${cool}"/>
        </linearGradient>
        <linearGradient id="ribbon" x1="0" x2="1">
          <stop offset="0%" stop-color="${primary}"/>
          <stop offset="100%" stop-color="${accent}"/>
        </linearGradient>
        <filter id="shadow" x="-30%" y="-30%" width="160%" height="160%">
          <feDropShadow dx="0" dy="14" stdDeviation="14" flood-color="#17345f" flood-opacity="0.16"/>
        </filter>
      </defs>
      <rect width="520" height="280" rx="28" fill="url(#learningBg)"/>
      <path d="M0 0h520v76H0z" fill="url(#ribbon)" opacity="0.9"/>
      <circle cx="448" cy="48" r="96" fill="#ffffff" opacity="0.14"/>
      <circle cx="72" cy="242" r="116" fill="${accent}" opacity="0.12"/>
      <path d="M34 212c70-48 134-42 194 18 28 28 58 32 92 12" fill="none" stroke="${primary}" stroke-w
idth="16" stroke-linecap="round" opacity="0.08"/>
      <rect x="38" y="42" width="236" height="176" rx="24" fill="#ffffff" opacity="0.74" filter="url(#
shadow)"/>
      <rect x="62" y="68" width="88" height="10" rx="5" fill="${warm}" opacity="0.64"/>
      <text x="82" y="122" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="24" font-
weight="900" fill="#17345f">${lineOne}</text>
      <text x="82" y="154" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="20" font-
weight="800" fill="#4c6382">${lineTwo}</text>
      <rect x="82" y="182" width="126" height="8" rx="4" fill="${primary}" opacity="0.16"/>
    </svg>
  `)

```



```

        <rect x="82" y="201" width="166" height="8" rx="4" fill="${accent}" opacity="0.18"/>
        ${motif}
      </svg>
    `)
  }

export const getExplicitResourceCoverUrl = resource => {
  const explicitCover =
    resource?.coverUrl ||
    resource?.cover_url ||
    resource?.thumbnailUrl ||
    resource?.thumbnail_url ||
    resource?.thumb_url ||
    ''
  return explicitCover ? resolveApiUrl(explicitCover) : ''
}

const getGeneratedResourceCoverUrl = resource => {
  const kind = resourceKind(resource)
  if (kind === 'image' && resource?.previewUrl) return resolveApiUrl(resource.previewUrl)

  const [primary, accent, paper] = palettes[hashText(resource?.doc_id || resource?.sourceId || resource?.title) % palettes.length]
  const rawTitle = resource?.title || resource?.filename
  const coverContext = { primary, accent, paper, rawTitle, resource, kind }

  return renderIllustratedResourceCover(coverContext)
}

const shouldUseExplicitCover = resource => {
  // 只有图片有真实预览图才用原图做封面，其余全部走生成
  const kind = resourceKind(resource)
  return kind === 'image' && Boolean(resource?.previewUrl)
}

export const getResourceCoverUrl = resource => {
  const explicitCover = getExplicitResourceCoverUrl(resource)
  if (explicitCover && shouldUseExplicitCover(resource)) return explicitCover

  return getGeneratedResourceCoverUrl(resource)
}

import { getGeneratedImage, getGeneratedResource, resolveApiUrl } from '../api/apis'
import { getResourceCoverUrl } from './resourceCover'

export const SAVED_GENERATED_RESOURCES_KEY = 'zhiban_saved_generated_resource_refs'

const readJson = key => {
  try {
    const value = JSON.parse(localStorage.getItem(key) || '[]')
    return Array.isArray(value) ? value : []
  }

```

```
    } catch {
      return []
    }
  }

const writeJson = (key, value) => {
  localStorage.setItem(key, JSON.stringify(value))
}

export const readSavedResourceRefs = visibility => {
  const refs = readJson(SAVED_GENERATED_RESOURCES_KEY)
  return visibility ? refs.filter(item => item.visibility === visibility) : refs
}

export const saveGeneratedResourceRef = payload => {
  const sourceId = String(payload.sourceId || payload.quizId || '')
  if (!sourceId) {
    throw new Error('生成资源缺少 id, 暂时不能保存。')
  }

  const kind = payload.kind || 'resource'
  const id = `${kind}-${sourceId}`
  const record = {
    id,
    sourceId,
    kind,
    fileType: payload.fileType || 'file',
    category: payload.category || 'reference',
    quizId: payload.quizId || '',
    title: payload.title || '',
    filename: payload.filename || '',
    content: payload.content || '',
    coverUrl: payload.coverUrl || '',
    previewUrl: payload.previewUrl || '',
    downloadUrl: payload.downloadUrl || '',
    annotations: Array.isArray(payload.annotations) ? payload.annotations : [],
    visibility: payload.visibility || 'private',
    createdAt: new Date().toISOString()
  }

  const refs = readSavedResourceRefs()
  const next = [record, ...refs.filter(item => item.id !== id)]
  writeJson(SAVED_GENERATED_RESOURCES_KEY, next)
  window.dispatchEvent(new CustomEvent('zhiban-generated-resource-saved', { detail: record }))
  return record
}

const unwrap = result => result?.data?.data ?? result?.data ?? result

const normalizeDetail = (detail, ref) => {
```

```

const item = unwrap(detail) || {}
const title =
  item.title ||
  item.filename ||
  item.file_name ||
  item.name ||
  ref.title ||
  ref.filename ||
  `${ref.kind === 'image' ? '生成图片' : '生成资源'}-${ref.sourceId}`
const fileType = item.file_type || item.fileType || item.resource_type || item.resourceType || ref.fileType || 'file'
const content = item.preview || item.content || item.text || item.preview_content || item.previewContent || ref.content || ''
const downloadUrl =
  item.download_url ||
  item.downloadUrl ||
  ref.downloadUrl ||
  item.url ||
  (ref.kind === 'image' ? `/image/${ref.sourceId}/download` : ref.quizId ? '' : `/resource/${ref.sourceId}/download`)
const previewUrl =
  item.preview_url ||
  item.previewUrl ||
  ref.previewUrl ||
  item.preview ||
  item.url ||
  item.image_url ||
  item.imageUrl ||
  ''

const resource = {
  doc_id: `${ref.kind}-${ref.sourceId}`,
  sourceId: ref.sourceId,
  source: 'generated',
  title,
  content,
  type: fileType,
  category: ref.category || (String(fileType).includes('exercise') ? 'exercise' : 'reference'),
  categoryLabel: '',
  visibility: ref.visibility || 'private',
  quizId: ref.quizId || '',
  created_at: item.created_at || item.createdAt || ref.createdAt || '',
  filename: ref.filename || title,
  annotations: Array.isArray(item.annotations) ? item.annotations : (Array.isArray(ref.annotations) ? ref.annotations : []),
  coverUrl: resolveApiUrl(item.cover_url || item.coverUrl || item.thumbnail_url || item.thumbnailUrl || ref.coverUrl || ''),
  previewUrl: resolveApiUrl(previewUrl),
  downloadUrl: resolveApiUrl(downloadUrl)
}

```

```
    return {
      ...resource,
      coverUrl: getResourceCoverUrl(resource)
    }
  }
}

export const hydrateSavedResourceRefs = async visibility => {
  const refs = readSavedResourceRefs(visibility)
  const settled = await Promise.allSettled(
    refs.map(async ref => {
      if (ref.quizId) return normalizeDetail({}, ref)
      const detail = ref.kind === 'image'
        ? await getGeneratedImage(ref.sourceId)
        : await getGeneratedResource(ref.sourceId)
      return normalizeDetail(detail, ref)
    })
  )

  return settled.map((item, index) => {
    if (item.status === 'fulfilled') return item.value
    return normalizeDetail({}, refs[index])
  })
}
```